



SKALA

3) Cloud

2. 컨테이너 이해 및 애플리케이션 컨테이너화

2026.7

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

과정 소개



교육 목표

컨테이너가 무엇인지 이해하고, 만든 프로그램을 직접 컨테이너로 만들어 실행해봅니다



교육 방식

하루에 이론 4시간 + 코드 리뷰로 진행하는
종합실습 2시간, 총 2일 과정입니다



필요한 사전 지식

기초 SW 수준이면 충분합니다. 컴퓨터·서버 전공
지식이 없어도 괜찮습니다



다음 과정

쿠버네티스 이해 및 애플리케이션 배포 —
오늘 만든 컨테이너를 다음 과정에서 더 크게 운영

이런 분들을 위한 과정입니다.~



*이번 기수에는 컴퓨터 전공이 아닌 분들도 많습니다.
그래서 이 과정은 처음부터 다르게 설계했습니다.*

터미널(까만 화면)을 한 번도 안 써보신 분도 괜찮습니다.

어려운 용어가 나올 때마다 우리 일상 속 비유로 먼저 설명하고, 그 다음에 정확한 뜻을 알려드립니다.

모르는 게 부끄러운 게 아니라, 질문 안 하는 게 아쉬운 겁니다 — 언제든 편하게 손을 들어주세요.

목표는 '전문가 되기'가 아니라 '두려움 없이 컨테이너를 다뤄보기'입니다.

2일 커리큘럼 한눈에 보기



하루 시간표 - 1일차, 2일차 동일

시간	구성	진행 방식
4시간	이론 + 미니 실습	개념 설명 → 바로 손으로 확인하는 짧은 실습 반복
2시간	종합실습 (코드 리뷰형)	직접 코드를 작성하고 같은 조와 함께 서로 코드 리뷰하면서 완성(제출은 개별로)

종합실습 시간에는 힌트와 체크리스트로 스스로 완성하게 할 겁니다~

준비물

도구	용도
터미널 (WSL 포함)	명령어를 입력하는 창
Docker Desktop / Engine	컨테이너를 만들고 실행하는 프로그램
텍스트 에디터 (VSCode 권장)	코드와 설정 파일을 작성하는 도구
편안한 마음!!!!!!!!!!!!!!	이게 가장 중요한 준비물임!!! 절대 @_@ 금지!

오늘 자주 만날 단어 미리보기

지금은 몰라도 됩니다. 이 과정이 끝나면 자연스럽게 익숙해집니다.
(저도 그랬습니다.)



컨테이너

프로그램을 담아 어디서든
똑같이 실행되게 하는 상자



이미지

컨테이너를 만드는 틀 (설계도)



Dockerfile

이미지를 만드는 방법을
적은 레시피



Registry

이미지를 보관하는 창고



Compose

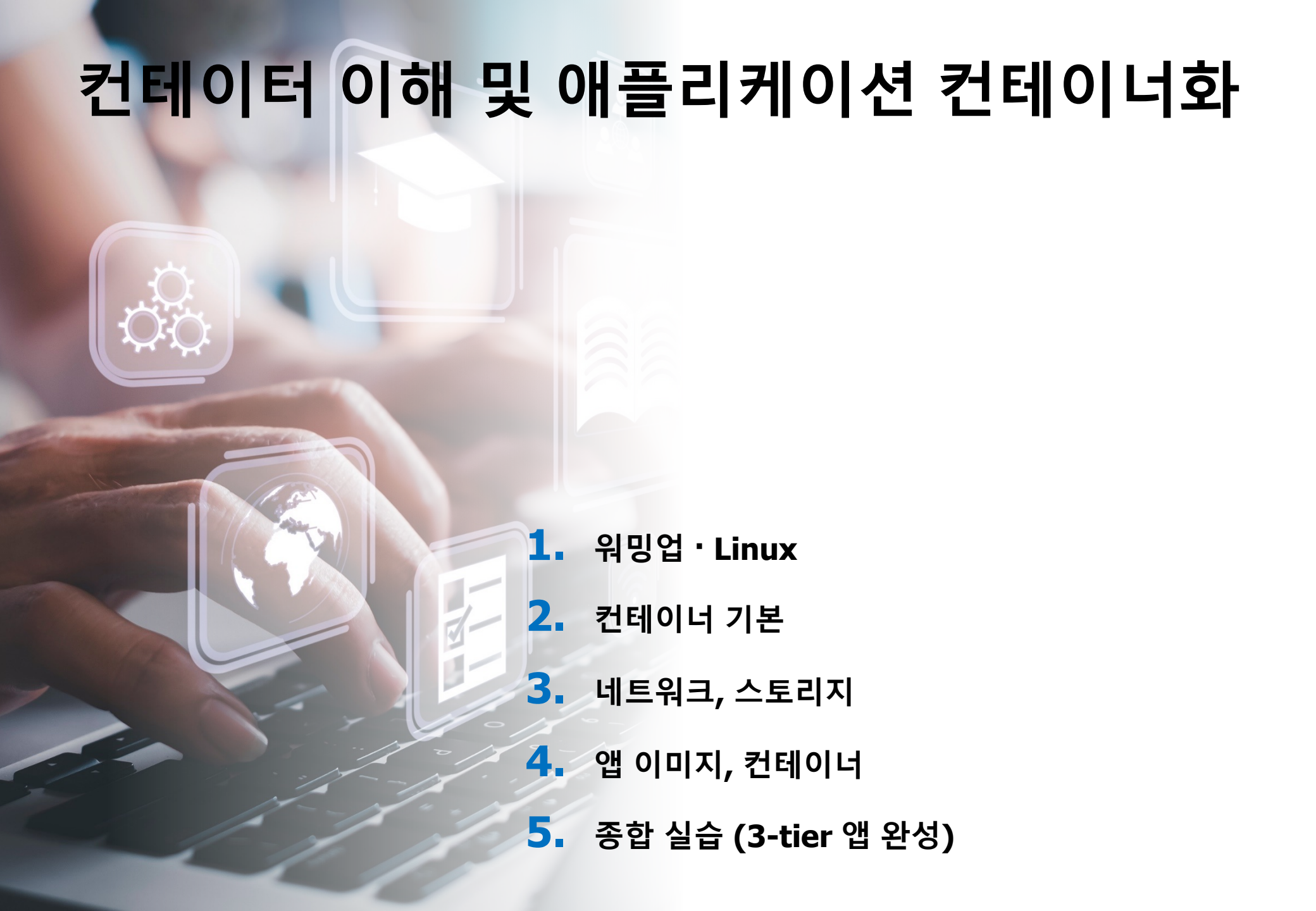
여러 컨테이너를 한 번에
지휘하는 도구



3-tier

화면, 서버, 저장소
세 부분으로 나눈 앱 구조

컨테이너 이해 및 애플리케이션 컨테이너화

- 
1. 워밍업 · Linux
 2. 컨테이너 기본
 3. 네트워크, 스토리지
 4. 앱 이미지, 컨테이너
 5. 종합 실습 (3-tier 앱 완성)

SKALA 3) Cloud – 2. 컨테이너 이해 및 애플리케이션 컨테이너화

0. 슬슬 달려볼까요?

- 진행하기 전에 간단하게~ 컴퓨터가 프로그램을 실행하는 원리를 아주 쉽게 짚고 갑니다

우리가 매일 쓰는 앱, 어떻게 실행될까



카카오톡을 열 때, 스마트폰 안에서는 무슨 일이 벌어질까요?

- 앱 아이콘을 누르면, 저장되어 있던 '**코드**'가 컴퓨터의 메모리 위에서 '**실행**'되기 시작
- 저장된 상태의 코드 $\neq$ 실행되고 있는 상태
- 이 둘의 차이를 이해하는 것이 컨테이너를 이해하는 첫걸음



프로그램 (Program)

“ 비유로 말하면

악보. 종이 위에 적혀있지만, 아직 연주되고 있지는 않은 상태 (연주하면 BTS의 Butter~)

☰ 정확히 말하면

컴퓨터에게 시킬 일을 미리 적어둔 파일로 저장 장치(디스크) 안에 가만히 있는 상태



프로세스 (Process)

“ 비유로 말하면

연주자가 악보를 보고 실제로 연주하고 있는 상태 (BTS Butter~~~)

☰ 정확히 말하면

프로그램이 실행되어, 컴퓨터의 메모리와 CPU를 사용하며 실제로 동작하고 있는 상태

프로세스



프로세스 (Process)

우리가 흔히 '앱을 켜다'고 말하는 순간이 바로,
프로그램이 프로세스가 되는 순간입니다.

“ 비유로 말하면

연주자가 악보를 보고 실제로 연주하고 있는 상태 (BTS Butter~~~)

☰ 정확히 말하면

프로그램이 실행되어, 컴퓨터의 메모리와 CPU를 사용하며 실제로 동작하고 있는 상태

운영체제 (OS)



운영체제 (OS)

“ 비유로 말하면

오케스트라의 지휘자 여러 연주자(프로세스)가 서로 부딪히지 않고 각자의 역할을 하도록 조율

☰ 정확히 말하면

컴퓨터의 하드웨어(CPU, 메모리, 디스크)와 프로그램 사이에서, 자원을 나눠주고 정리

Windows, macOS, 그리고 오늘 배울 Linux가 모두 운영체제



서버 (Server)

“ 비유로 말하면

24시간 문을 여는 편의점. 손님(사용자 요청)이 언제 오든 응대할 준비 완료

☐ 정확히 말하면

인터넷에 연결된 채 항상 켜져 있으면서, 다른 컴퓨터의 요청을 받아 응답해주는 컴퓨터

우리가 오늘 만들 컨테이너도, 결국 이 서버 위에서 실행

오늘과 내일, 우리가 만들 것 미리보기

 **결승선을 먼저 보고 출발하면 길을 잃지 않습니다. (가는 길은 여러 가지)**

Day 1 에는

화면(Web) + 서버(WAS) + 저장소(DB), 세 부분으로 이루어진 미니 게시판을 컨테이너로 직접 실행

Day 2 에는

그 세 부분을 'Docker Compose'라는 도구로 한 번에 실행하도록 완성



옆자리 동료와 이야기해보기

프로그램과 프로세스의 차이에 대해 다른 비유로 옆 사람에게 설명해보세요.



2분

[Quiz]

0. 슬슬 달려볼까요?

저장 장치 안에 가만히 있는 코드를 무엇이라고 부를까요?

- 프로세스 (실행 중인 상태)
- 프로그램 (저장되어 있는 상태)
- 서버 (항상 켜진 컴퓨터)
- 운영체제 (자원을 나눠주는 소프트웨어)

[Quiz]

0. 슬슬 달려볼까요?

저장 장치 안에 가만히 있는 코드를 무엇이라고 부를까요?

- ☐ 프로세스 (실행 중인 상태)
- ☒ 프로그램 (저장되어 있는 상태)
- ☐ 서버 (항상 켜진 컴퓨터)
- ☐ 운영체제 (자원을 나눠주는 소프트웨어)

💡 프로그램이 실행되기 시작하면 그때부터 '프로세스' 컨테이너를 설명할 때 다시 나옵니다!

SKALA 3) Cloud – 2. 컨테이너 이해 및 애플리케이션 컨테이너화

1. Linux OS 기초

- 컨테이너와 대화를 위해 터미널과 친해지기

왜 서버는 Linux를 쓸까

내 컴퓨터는 *Windows*인데, 왜 굳이 *Linux*를 배워야 할까요?

- 전 세계 서버의 대부분이 Linux 위에서 실행 중
→ 가볍고, 안정적이고, 무료
- 결정적으로, 오늘 배울 '컨테이너'는 사실 **Linux**의 기술을 빌려 만들어진 것
→ 컨테이너를 이해하려면 Linux의 기본 동작 방식을 알아야 함

터미널 (Terminal)



터미널 (Terminal)

“ 비유로 말하면

마우스로 그림을 그리는 대신, 컴퓨터에게 글자로 직접 말을 거는 대화창

☰ 정확히 말하면

마우스 클릭 없이, 글자로 된 명령어를 입력해 컴퓨터를 조작하는 화면
검은 화면에 흰 글씨, 낯설지만 괜찮아~

터미널 (Terminal)

마우스 클릭 vs 터미널 명령어

둘 다 '컴퓨터에게 시키는 방법'



마우스로 클릭하기

- 눈에 보이는 아이콘을 클릭
- 직관적이고 배우기 쉬움
- 같은 작업을 반복하려면 매번 클릭
- 정확히 같은 결과를 재현하기 어려움



터미널에 타이핑하기

- 글자로 된 명령어를 입력
- 처음엔 낯설지만 빠르고 정확함
- 같은 명령어를 저장해두면 재사용 가능
- 서버 컴퓨터를 다룰 때는 사실상 필수

리눅스의 파일 정리 방식 — 나무 이야기

 ***Windows의 '내 PC, C드라이브' 대신, 리눅스는 뿌리(root) 하나에서 시작***

- / (루트) — 나무의 뿌리, 모든 것이 여기서 시작
 - /home — 나뭇가지 하나, 나만의 개인 폴더
 - /etc — 또 다른 가지, 설정 파일들이 모인 곳
- 지금은 몰라도 됩니다 — 자연스럽게 이 나무 구조에 익숙

pwd



pwd

“ 비유로 말하면

"저 지금 어디 있어요?"라고 컴퓨터에게 묻는 것과 같습니다.

☰ 정확히 말하면

Print Working Directory의 줄임말
지금 내가 서있는(작업 중인) 폴더의 위치

\$ pwd → /home/student 처럼 현재 위치가 출력



ls

“ 비유로 말하면

"이 방 안에 뭐가 있어요?"

☰ 정확히 말하면

list의 줄임말. 지금 폴더 안에 어떤 파일과 폴더가 있는지 보여줍니다.

\$ ls -al → 숨겨진 파일까지 자세히 보여줍니다.

cd



cd

“ 비유로 말하면

"다른 방으로 이동하실게요"

☰ 정확히 말하면

Change Directory의 줄임말. 폴더를 이동할 때 사용

\$ cd Documents → Documents 폴더로 이동 / \$ cd .. → 한 칸 상위 폴더로 이동

친하게 지내기2~



옆자리 동료와 이야기해보기
pwd, ls, cd를 각자 5번씩 입력해보고,
어떤 결과가 나왔는지 옆 사람과 비교해보세요.



2분



mkdir

“ 비유로 말하면

새 서랍을 만드는 것과 같습니다.

☰ 정확히 말하면

make directory의 줄임말. 새 폴더 생성

\$ mkdir project



touch

“ 비유로 말하면

서랍 안에 빈 종이 한 장을 넣어두는 것

☰ 정확히 말하면

내용이 없는 빈 파일을 새로 만듭니다.

\$ touch app.py

cp

**cp**

“ 비유로 말하면

서류를 복사기에 넣는 것

☰ 정확히 말하면

copy의 줄임말. 파일을 복사

\$ cp a.txt b.txt

mv



mv

“ 비유로 말하면

서류를 다른 서랍으로 옮기거나, 새 이름표를 붙이는 것과 같습니다.

☰ 정확히 말하면

move의 줄임말. 파일을 이동하거나 이름을 바꿉니다.

\$ mv a.txt archive/

rm



rm

“ 비유로 말하면

종이를 파쇄기에 넣는 것과 같습니다 — 한번 넣으면 되돌릴 수 없습니다.

☰ 정확히 말하면

remove의 줄임말. 파일이나 폴더를 삭제합니다. 휴지통을 거치지 않고 완전히 사라집니다.

주의! `rm -r` 폴더명 을 실행하기 전, 정말 지을 폴더가 맞는지 꼭 한번 더 확인하세요.



cat

“ 비유로 말하면

편지를 처음부터 끝까지 다 읽는 것과 같습니다.

☰ 정확히 말하면

파일의 전체 내용을 화면에 보여줍니다.

\$ cat app.log



tail

“ 비유로 말하면

편지의 마지막 몇 줄만 훑어보는 것과 같습니다.

☰ 정확히 말하면

파일의 마지막 부분만 보여줍니다. 최근 로그를 확인할 때 자주 씁니다.

\$ tail -n 20 app.log

grep



grep

“ 비유로 말하면

긴 책에서 원하는 단어가 있는 페이지만 찾아주는 것과 같습니다.

☰ 정확히 말하면

파일이나 결과 안에서 원하는 문자열을 검색합니다.

\$ cat app.log | grep ERROR

권한이란 무엇인가 — 열쇠 이야기

 모든 파일에는 '누가 열어볼 수 있고, 누가 고칠 수 있는지' 정해진 열쇠 규칙

- 읽기(**r**) · 쓰기(**w**) · 실행(**x**), 세 가지 권한
- 이 열쇠는 소유자 / 같은 그룹 / 그 외 모든 사람, 이렇게 세 그룹에 각각 따로 적용
- 스크립트 파일을 실행하려면 '실행 권한'이라는 열쇠가 필요

chmod



chmod

“ 비유로 말하면

문에 새 자물쇠를 달아, 누가 들어올 수 있는지 바꾸는 것과 같습니다.

☰ 정확히 말하면

change mode의 줄임말. 파일의 권한(읽기·쓰기·실행)을 바꿔줍니다.

\$ chmod +x deploy.sh → deploy.sh 파일에 실행 권한을 추가

지금 컴퓨터 안에서 일하는 사람들 — 프로세스

 *컴퓨터 안에는 지금 이 순간에도 여러 프로그램이 동시에 '일'*

- **ps -ef** 는 지금 일하고 있는 모든 프로세스의 명단을 보여줌
- 너무 오래 일하거나 문제가 생긴 프로세스는 kill 명령어로 멈출 수 있음
- 이 개념은 잠시 후 '컨테이너 = 프로세스'라는 것을 이해할 때 다시 등장

셸 스크립트란 — 할 일 목록 자동화

 명령어를 매번 하나씩 입력하기 귀찮다면, 순서대로 적어서 한 번에 실행

- 이렇게 명령어를 모아놓은 파일을 '셸 스크립트' (확장자 .sh)
- 변수 · 조건문 · 반복문, 이 세 가지만 알아도 충분

변수 — 이름표를 붙여두기

자주 바뀌는 값에 이름을 붙여두면 나중에 고치기 편합니다.



```
#!/bin/bash
```

```
APP_NAME="my-app"
```

```
echo "안녕하세요, $APP_NAME 입니다"
```



APP_NAME이라는 이름표에 "my-app"이라는 값을 붙여둔 것뿐

조건문 — 만약에 ~라면

"만약 우산이 있으면 밖에 나간다"와 같은 논리



```
#!/bin/bash
```

```
if [ -f "app.py" ]; then
    echo "app.py 파일이 있어요"
else
    echo "app.py 파일이 없어요"
fi
```

💡 대괄호 [] 양옆에는 꼭 띄어쓰기를 해주세요 — 가장 흔한 실수

반복문 — 같은 일을 여러 번

"1번부터 3번까지 순서대로 인사하기"와 같은 논리



```
#!/bin/bash
```

```
for i in 1 2 3; do
```

```
    echo "$i 번째 인사입니다"
```

```
done
```


[실습] 나만의 폴더 만들어보기

🕒 오늘 배운 명령어를 순서대로 사용해 실습 폴더를 준비합니다.

- ❶ cd 로 홈 폴더로 이동한다
- ❷ mkdir workspace 로 폴더를 만든다
- ❸ cd workspace 로 이동한다
- ❹ touch app.py 로 빈 파일을 만든다
- ❺ ls -al 로 결과를 확인한다

[Quiz]

파일에 실행 권한을 추가하고 싶을 때 쓰는 명령어는?

- ☐ ls -al
- ☐ cd ..
- ☐ chmod +x 파일명
- ☐ rm -r 파일명

[Quiz]

파일에 실행 권한을 추가하고 싶을 때 쓰는 명령어는?

☐ ls -al

☐ cd ..

☒ **chmod +x 파일명**

☐ rm -r 파일명

 **chmod +x** 는 파일에 '실행'이라는 열쇠를 추가하는 명령어

오늘 배운 명령어, 한 장으로 총정리

명령어	한 줄 뜻
pwd / ls / cd	위치 확인 / 목록 보기 / 이동
mkdir / touch	폴더 만들기 / 파일 만들기
cp / mv / rm	복사 / 이동, 이름 변경 / 삭제
cat / tail / grep	전체 읽기 / 끝부분 읽기 / 검색
ps / kill	프로세스 확인 / 종료

[Quiz]

파일 안에서 '**ERROR**'라는 글자를 찾고 싶을 때 쓰는 명령어는?

☐ mkdir

☐ grep

☐ chmod

☐ mv

[Quiz]

파일 안에서 'ERROR'라는 글자를 찾고 싶을 때 쓰는 명령어는?

☐ mkdir

☒ **grep**

☐ chmod

☐ mv

💡 **grep**은 긴 텍스트 안에서 원하는 단어를 찾아주는 검색 명령어입니다.

Linux, 이것만은 기억하세요

✓ 터미널은 컴퓨터와 글자로 대화하는 창

✓ `mkdir`, `touch`, `cp`, `mv`, `rm` 파일 작업

✓ 셸 스크립트는 변수, 조건문, 반복문 세 가지가 핵심

✓ `pwd`, `ls`, `cd` 로 위치를 확인하고 이동

✓ `chmod`로 파일의 실행 권한 변경 가능

✓ 컨테이너는 리눅스 기초 기반으로 진행

SKALA 3) Cloud – 2. 컨테이너 이해 및 애플리케이션 컨테이너화

2. 컨테이너 기본

- 비유로 먼저 이해하고, 그 다음에 손으로 직접 만들어봅니다

생각해보기 — 이사할 때 이야기



이사를 가는 세 가지 방법을 상상해봅시다.

- 트럭 하나에 살림살이를 그냥 다 쏟아붓고 옮긴다 (뒤죽박죽, 깨지기 쉬움)
- 방마다 큰 박스 하나씩 통째로 옮긴다 (안전하지만 박스가 무겁고 큼)
- 물건마다 딱 맞는 작은 상자에 잘 포장해서 옮긴다
(가볍고, 어디서든 바로 풀어 쓸 수 있음)
- 이 세 가지가 각각 **Bare-metal, VM, Container**와 비슷

옛날 방식 — Bare-metal (내 방에 다 담기)



컴퓨터 한 대에 프로그램을 그냥 직접 설치해서 쓰는 방식

- 한 방에 여러 사람의 짐을 그냥 다 섞어놓은 것
- 프로그램 **A**와 **B**가 서로의 물건(라이브러리)을 밟고 지나가며 충돌
- 컴퓨터 한 대가 고장 나면, 그 안의 모든 프로그램이 한꺼번에 멈춥니다.

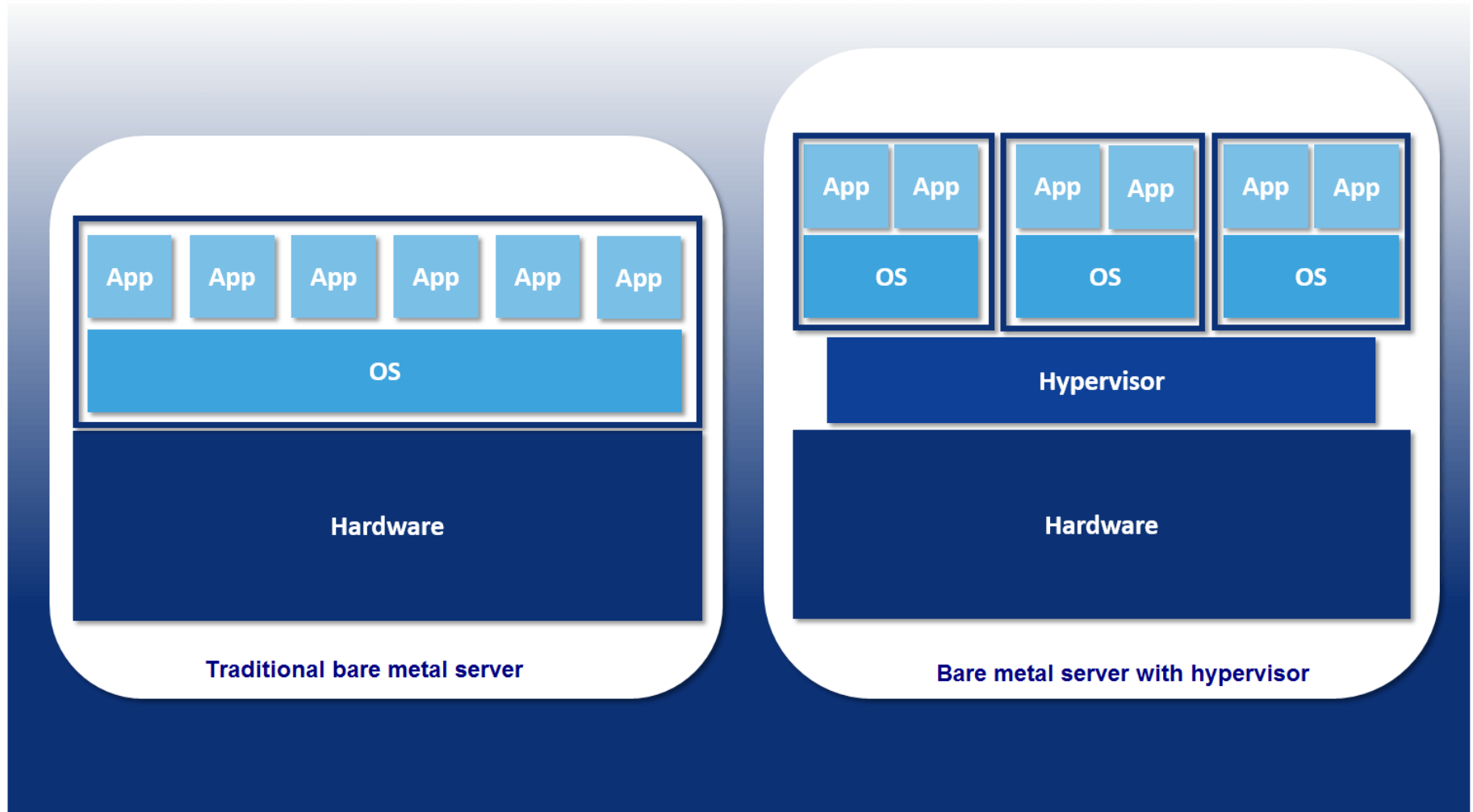
옛날 방식 — Bare-metal (내 방에 다 담기)



옛날 방식 — Bare-metal (내 방에 다 담기)



옛날 방식 — Bare-metal (내 방에 다 담기)



옛날 방식 — Bare-metal (내 방에 다 담기)

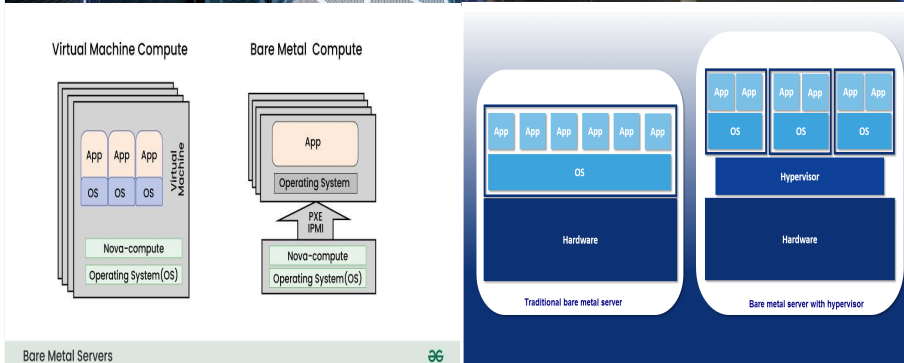


【 It means... 】

Bare-metal은 운영체제(OS)를 바로 설치해 사용하던
물리 서버

"철판 위에(OS를 없어)" 바로 **Application** 사용

- 여러 개의 Rack으로 구성된 데이터 센터 내 공간에서
- Rack 안에 작은 단위의 서버 여러 개가 동작



옛날 방식 — Bare-metal (내 방에 다 담기)



문제점 1: 한 서버에 여러 앱을
함께 돌리기 어려움

- 한 서버에
Django+Python 앱과
Spring+Java 앱을 동시에
올리면?
→ 라이브러리 버전 충돌
→ 자바와 파이썬이
CPU 메모리를 서로
잡아먹음
→ 하나가 죽으면
서버 전체가 불안정
- 달걀 한 바구니 흔들기

문제점 2: 환경 설정이 지옥

- 앱 하나 배포 시
→ 라이브러리 설치
→ **OS** 설정 변경
→ 방화벽 수정
→ 사용자가 직접 모든 설정 체크
- **20대 서버 = 20번 반복**
(자동화는...)

"환경 세팅이 개발자의 시간을 잡아
먹는 괴물"

문제점 3: 장애 발생 시, 서버 자체가
죽음

- 물리 서버 전원·디스크·**RAM** 문제
→ 바로 전체 서비스 중단
- 복구 시간
→ 보통 수 시간 ~ 수십 시간
- **24시간 서비스 가능**
but...

"서버 1대가 죽으면 회사가 죽는다."

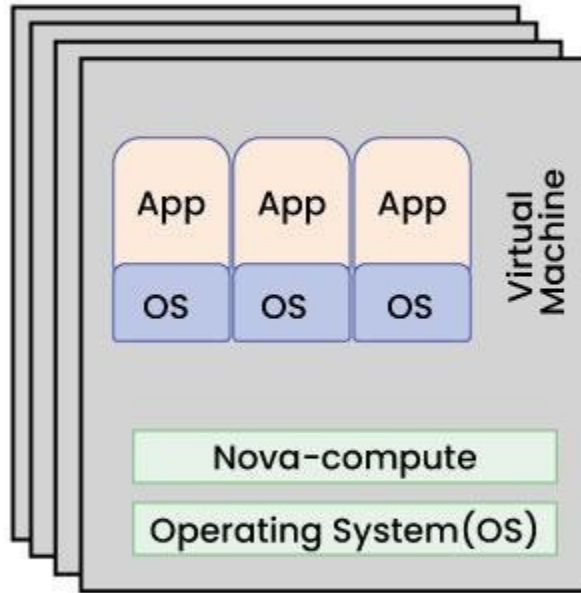
그 다음 방식 — VM (건물을 나눠 원룸 주기)

 **한 건물(컴퓨터)을 여러 개의 독립된 원룸(가상 컴퓨터)으로 나누는 방식**

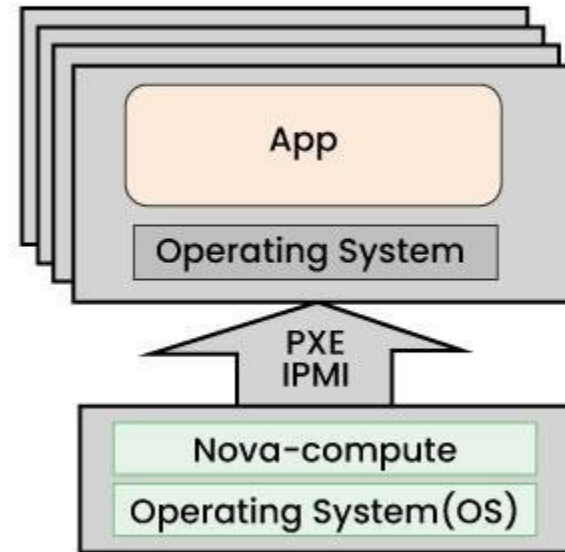
- 각 원룸은 완전히 독립되어 있어 옆방에 영향을 주지 않습니다.
- 하지만 원룸마다 부엌, 화장실을 통째로 새로 짓는 것과 같아서 무겁고, 짓는 데 시간이 오래 걸립니다.

그 다음 방식 — VM (건물을 나눠 원룸 주기)

Virtual Machine Compute



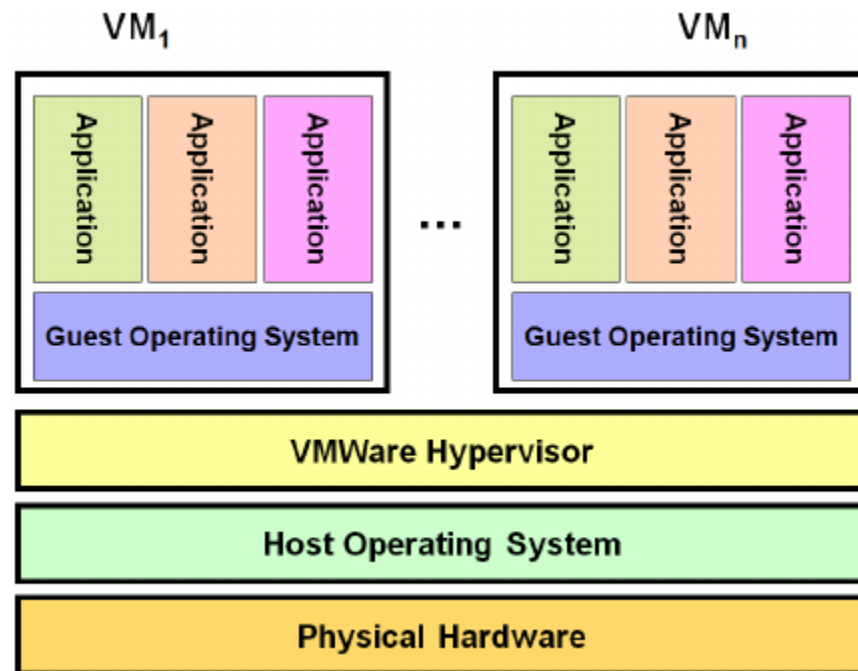
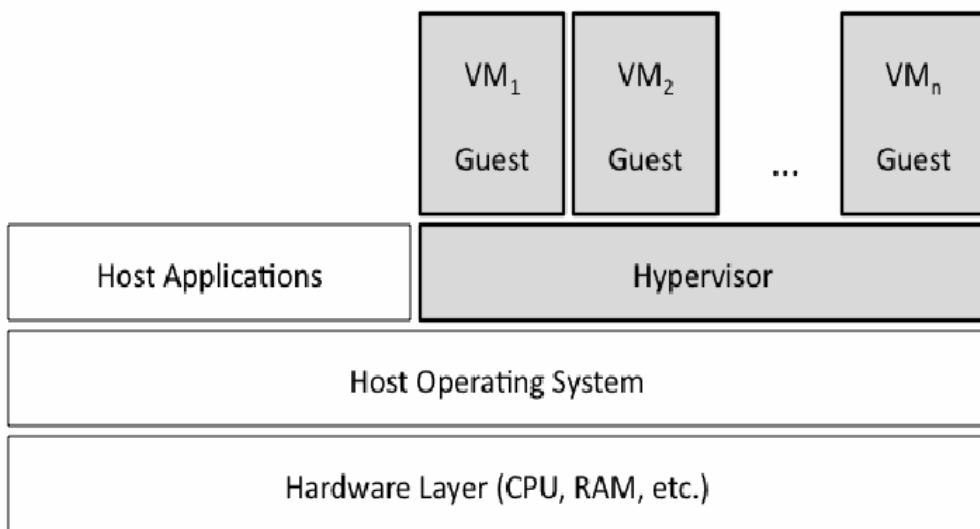
Bare Metal Compute



Bare Metal Servers

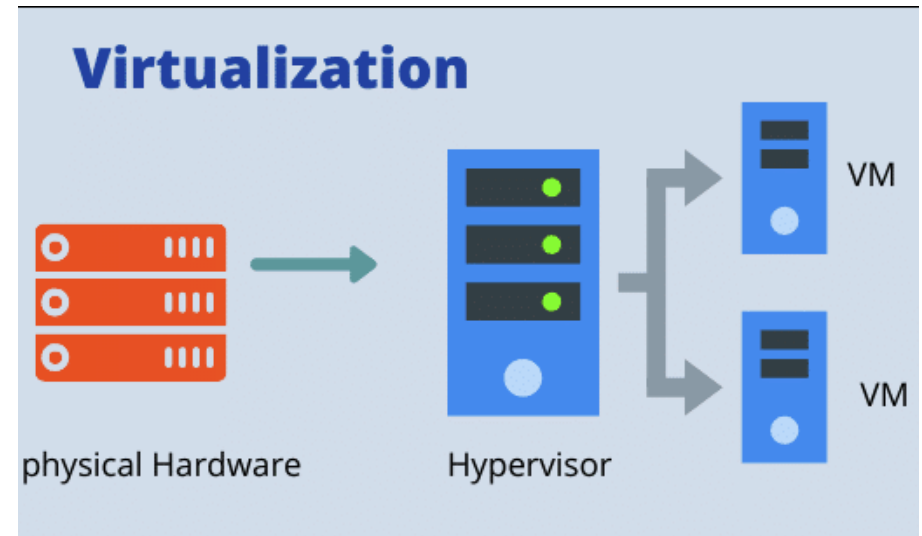
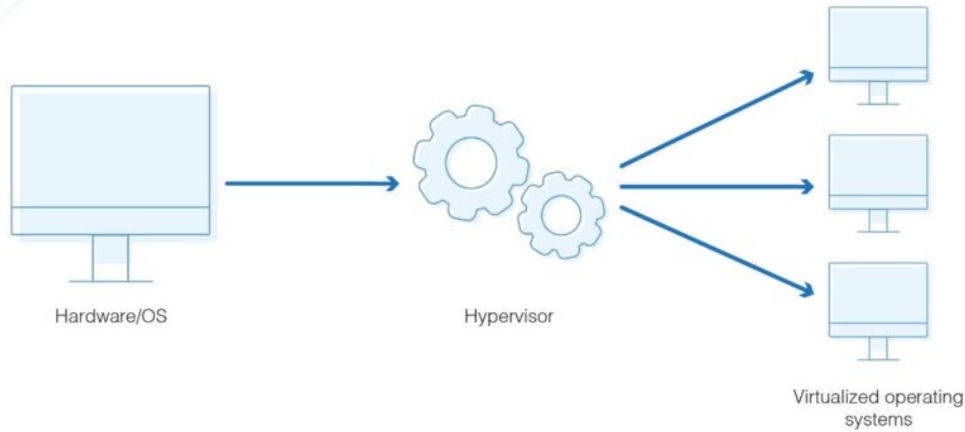


VM(Virtual Machine) 시대 – 혁명처럼 등장했지만 한계도 존재



VM(Virtual Machine) 시대 – 혁명처럼 등장했지만 한계도 존재

What Is a Hypervisor?



VM(Virtual Machine) 시대 – 혁명처럼 등장했지만 한계도 존재

【 VM이란? 】

- ✓ 하드웨어 위에 **Hypervisor (VM 관리 프로그램)** 설치
- ✓ 그 위에 가상의 컴퓨터(**Guest OS**) 여러 개 생성
- ✓ 각 **VM**은 독립된 컴퓨터처럼 행동
 - 자체 OS
 - 자체 라이브러리
 - 자체 파일 시스템

구분	설명
장점	• 완전한 격리 - VM 1: Ubuntu + Python - VM 2: CentOS + Java - VM 3: Windows + .NET • 모두 서로 간섭 없이 잘 돌아감 → 최고의 안정성
단점	• 무거움 (OS 전체를 올려야 함) → 성능이 반토막나는 느낌 - VM 하나 = OS 1개 + 앱OS 자체가 무거움 - 긴 부팅 시간 (VM 1개 실행 시 1~2분 소요) • 리소스 낭비 심함 - VM마다 OS가 따로 있어 RAM/CPU 소모 심함 - 쓰지 않는 OS 프로세스도 자원을 계속 사용 - 서버 1대에 100개 VM 돌리는 건 사실상 불가 • 느린 개발/운영 속도 - VM 이미지 만들기 → 수 분 실행/배포 → 수 분 확장(scale-out) → 느림
	• 트래픽 급증 시 대응이 어려움

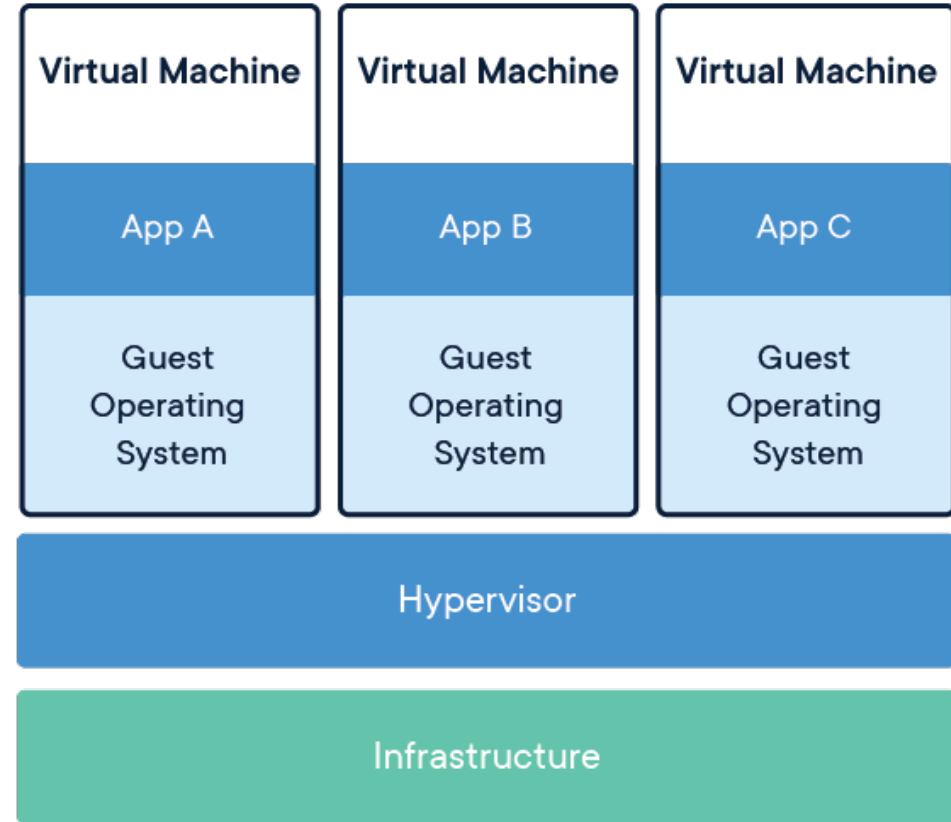
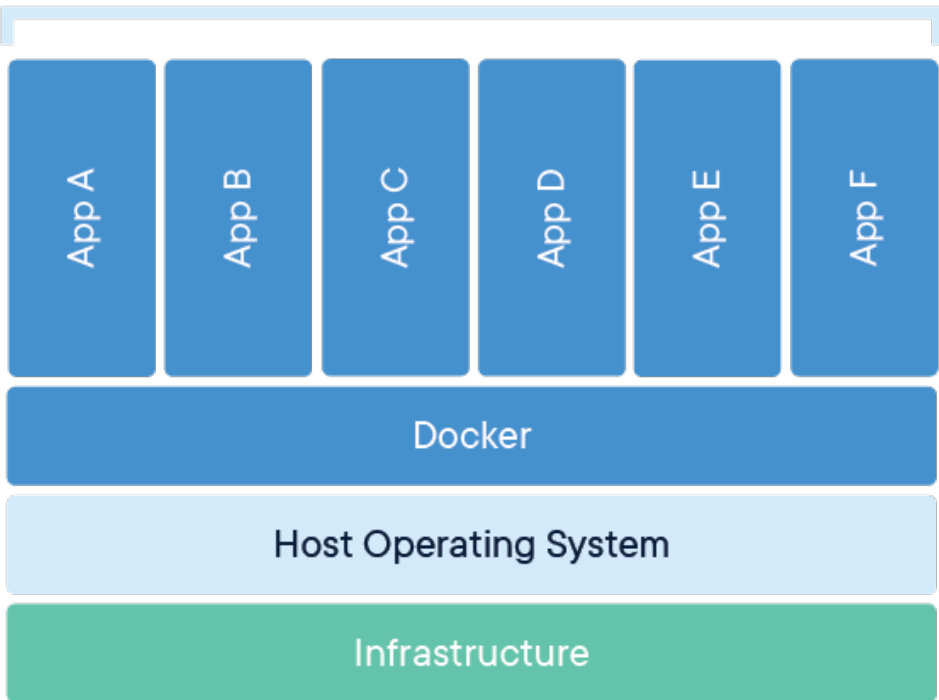
지금 방식 — Container (도시락 이야기)

 **건물의 공용 주방(운영체제)은 같이 쓰되, 도시락(컨테이너)만 따로**

- 건물 전체를 다시 짓지 않아도 되니, 훨씬 가볍고 빠릅니다.
- 도시락 하나 하나는 서로 섞이지 않게 잘 포장되어 있어,
내용물이 서로 영향을 주지 않음
 - ‘컨테이너’ 개념

컨테이너(Container) 시대 – 앱 배포의 게임 체인저

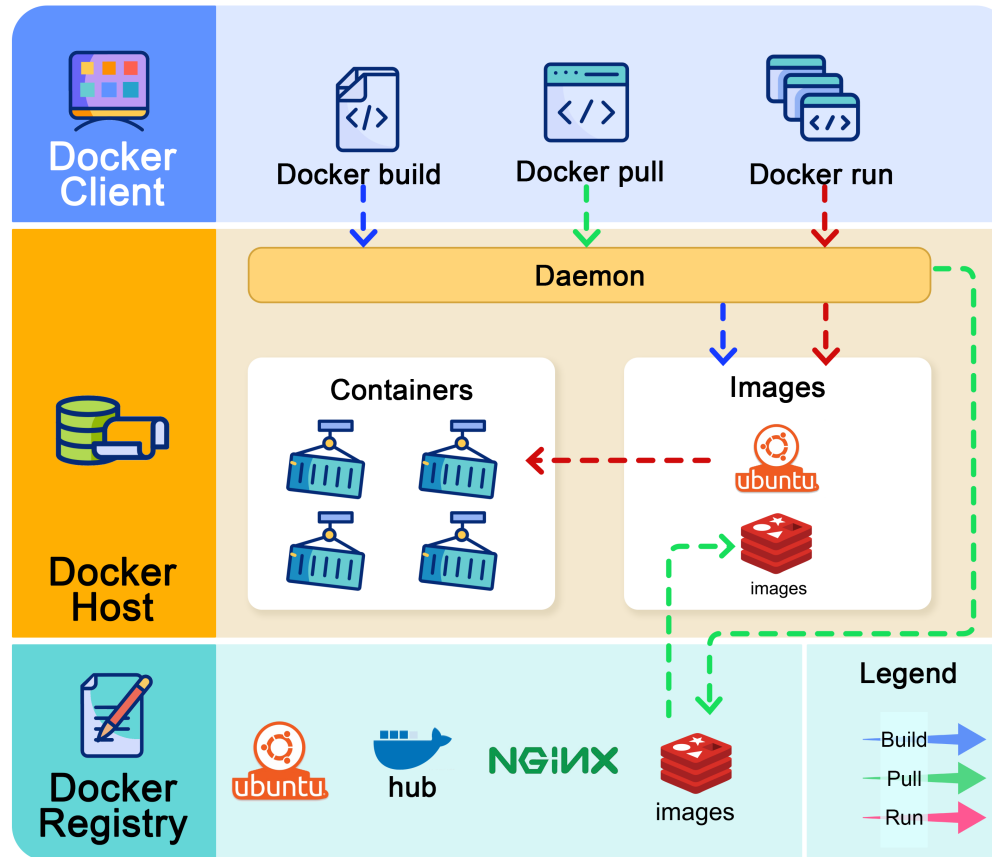
Containerized Applications



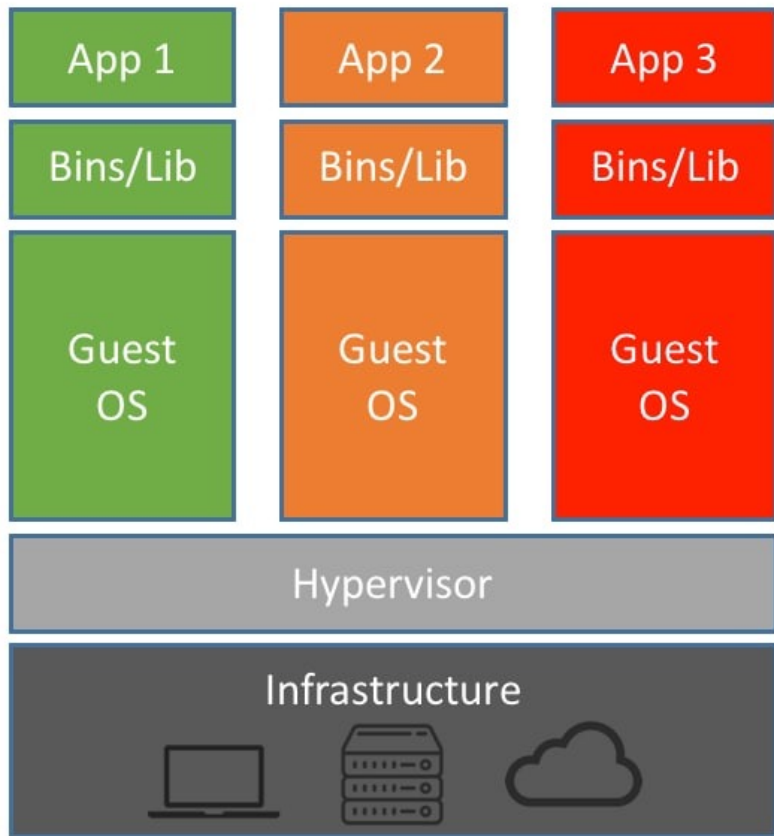
컨테이너(Container) 시대 – 앱 배포의 게임 체인저

How does Docker Work ?

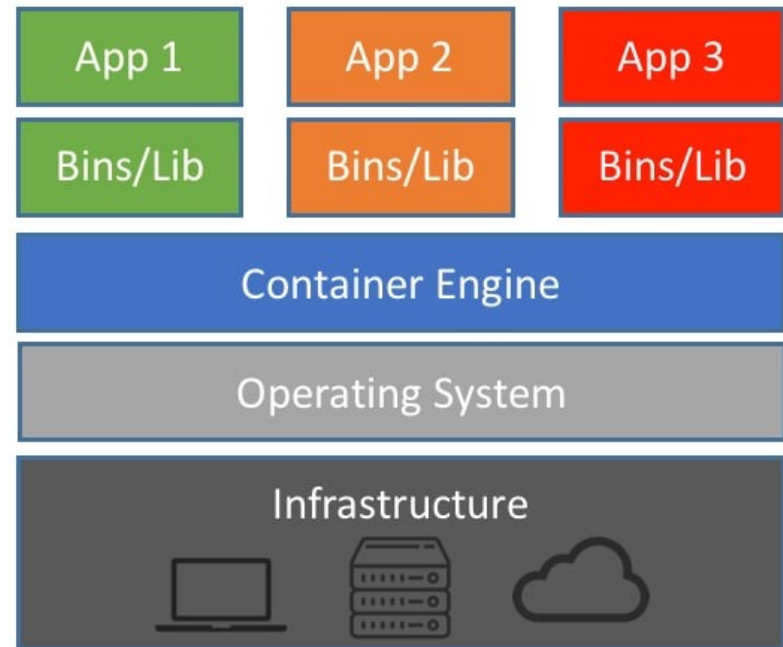
 blog.bytebytego.com



컨테이너(Container) 시대 – 앱 배포의 게임 체인저

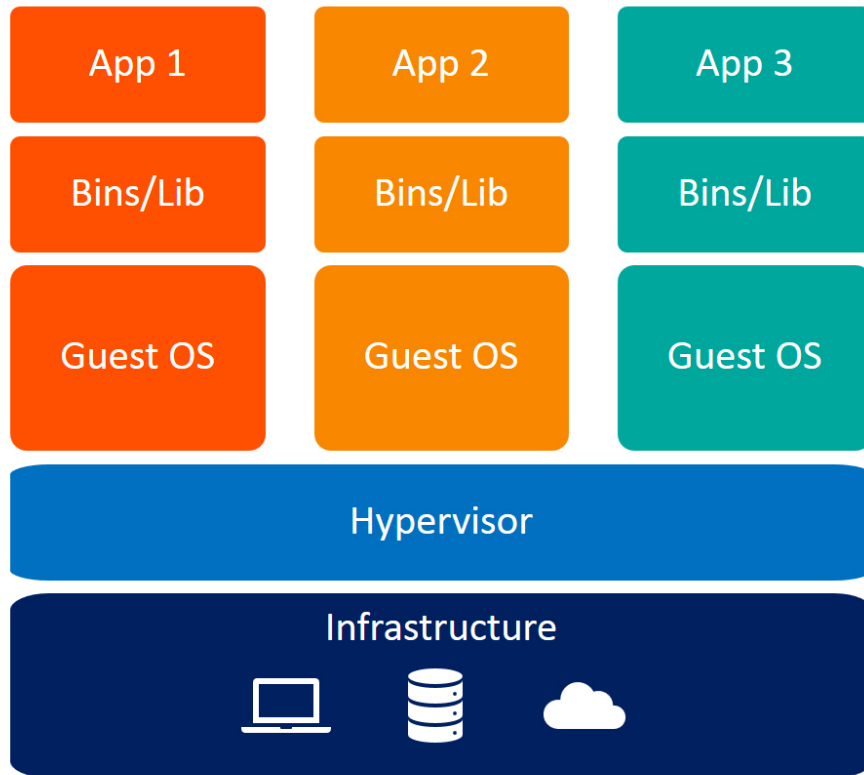


Machine Virtualization

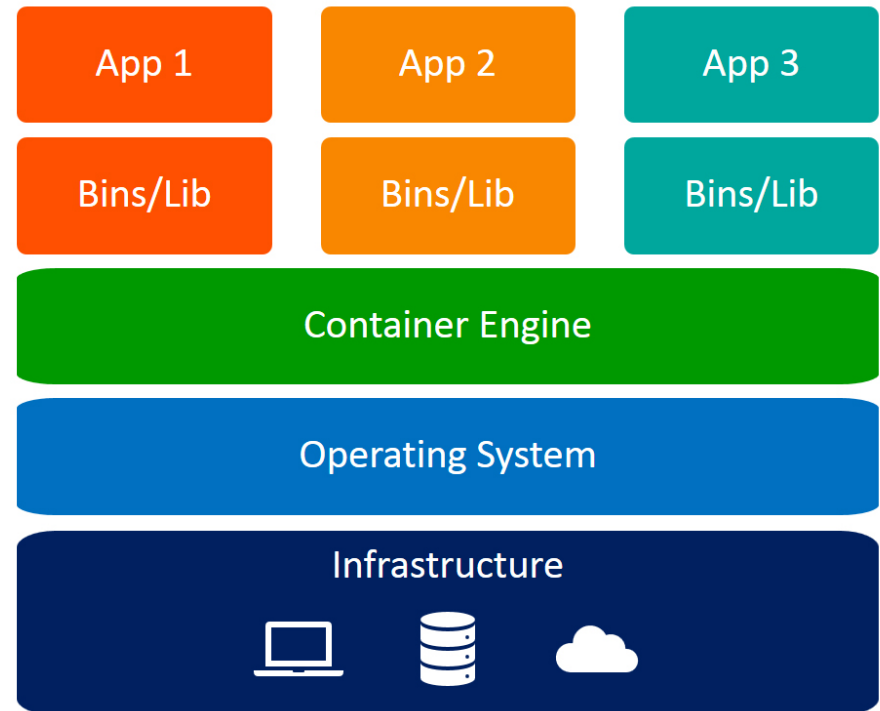


Containers

컨테이너(Container) 시대 – 앱 배포의 게임 체인저



Virtual Machines



Containers

컨테이너(Container) 시대 – 앱 배포의 게임 체인저

【 컨테이너란? 】

- ✓ OS 커널을 공유하면서
- ✓ 앱 실행에 필요한 라이브러리, 설정만 묶어서 패키징한 것
(**Docker 이미지**)
 - VM = 무거운 집 전체
 - Container
= 필요시 빌려 쓰는 게스트하우스

장점

구분	설명
장점	• VM보다 가벼움 - 커널(OS)은 공유라이브러리, 실행 파일만 포함 → 용량 매우 작음(수백 MB → 수십 MB) • 어디서든 동일하게 실행 - Docker file 하나로 모든 환경을 코드로 정의 - 개발자 Mac / 운영 Linux / 클라우드 등 어디서나 동일하게 실행 - "작동 안 해요... 제 PC에서는 되는데요?"가 사라질 수 있음 • 빠른 생성/삭제 - 컨테이너 실행: 0.1~1초 / VM 실행: 30~60초 → 자동 확장 & 자동 복구에 최적화 • 마이크로서비스와 잘 맞음 - 기능별로 작은 컨테이너로 분리 - 독립적으로 배포 및 확장 - 현대적 서비스 구조(MSA)의 기본 요소

컨테이너(Container) 시대 – 앱 배포의 게임 체인저

【 컨테이너란? 】

- ✓ OS 커널을 공유하면서
- ✓ 앱 실행에 필요한 라이브러리, 설정만 묶어서 패키징한 것
(**Docker 이미지**)
 - VM = 무거운 집 전체
 - Container
= 필요시 빌려 쓰는 게스트하우스

구분	설명
한계	• 단일 서버에서 컨테이너 10개, 20개는 OK • 만약, 수 백 개, 수 천 개 시 운영 시는 아래 문제 발생 빈도 ↑ - 어떤 서버에 어떤 컨테이너가 올라가야 하는지 - 컨테이너 다운 시 재실행 - 트래픽 증가 시 자동으로 컨테이너 수 증가 처리 - 설정(네트워크, 스토리지) 자동화 여부?

컨테이너가 많아질수록 오히려 운영이 어려움 증가

세 가지 방식, 한눈에 비교



Bare-metal

- 한 방에 다 같이 (격리 없음)
- 충돌 위험이 큼
- 설치·설정에 시간이 오래 걸림



VM → Container

- VM: 건물을 통째로 나눔 (무거움)
- Container: 도시락만 따로 (가벼움)
- 지금은 대부분 Container를 사용

컨테이너 (Container)



컨테이너 (Container)

“ 비유로 말하면

잘 포장된 도시락. 어디로 가져가든 뚜껑만 열면 바로 먹을 수 있습니다.

☐ 정확히 말하면

프로그램이 실행되는 데 필요한 모든 것(코드, 라이브러리, 설정)을 하나로 포장해, 어떤 컴퓨터에서든 똑같이 실행되게 만든 상자입니다.

그래서 "내 컴퓨터에서는 되는데요?" 라는 문제가 컨테이너를 쓰면 크게 줄어듭니다.

이미지 vs 컨테이너



이미지 vs 컨테이너

“ 비유로 말하면

붕어빵 틀과 붕어빵의 관계

틀(이미지)은 하나지만, 그 틀로 붕어빵(컨테이너)을 여러 개 구워낼 수 있습니다.

☐ 정확히 말하면

이미지는 '실행되기 전, 저장된 설계도'이고,
컨테이너는 '그 이미지로 실제 실행되고 있는 것'입니다.

아까 배운 프로그램(이미지) vs 프로세스(컨테이너)와 정확히 같은 관계!

친하게 지내기3~



옆자리 동료와 이야기해보기

이미지와 컨테이너의 차이를, 다른 비유를 써서 옆 사람에게 설명해보세요.



2분



Docker

“ 비유로 말하면

붕어빵 틀을 만들고, 굽고, 포장까지 도와주는 만능 기계

☰ 정확히 말하면

컨테이너를 쉽게 만들고, 실행하고, 관리할 수 있게 도와주는 프로그램(도구)
우리는 오늘부터 이 도구를 사용합니다.

[실습] Docker 설치

🕒 실습 전, 내 컴퓨터에 **Docker**를 설치합니다.

- 1 스크립트 파일을 다운로드하여 원하는 위치(예: 바탕화면, 다운로드 폴더)에 저장
- 2 터미널(Terminal) 앱 실행
- 3 `cd ~/Downloads` (파일을 저장한 폴더로 이동)
`chmod +x skala-docker-setup.sh`
`./skala-docker-setup.sh`
- 4 스크립트가 안내하는 메시지를 그대로 따라가면 됩니다
- 5 중간에 macOS 관리자 비밀번호 입력창이 뜨면 본인 Mac 로그인 비밀번호를 입력
- 6 스크립트 종료 후, 안내에 따라 Docker 앱을 한 번 직접 실행 (`open -a Docker`)

[실습] Docker 설치 확인하기

🕒 실습 전, 내 컴퓨터에 **Docker**가 잘 준비되어 있는지 확인합니다.

- ❶ 터미널을 연다
- ❷ `docker --version` 을 입력해 버전이 나오는지 확인한다
- ❸ `docker run hello-world` 를 입력해 간단한 테스트 컨테이너를 실행해본다
- ❹ Hello from Docker! 메시지가 보이면 성공입니다

[실습] Docker 설치 확인하기

```
[→ slack docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
58dee6a49ef1: Pull complete
c3bdf82c34d1: Download complete
Digest: sha256:5dd0d3e6e255913fc30f90b9f2b1d359cc2cbdb48090cc4b65f1676e203243cc
Status: Downloaded newer image for hello-world:latest
```

Hello from Docker!

This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(arm64v8)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:

```
$ docker run -it ubuntu bash
```

Share images, automate workflows, and more with a free Docker ID:

<https://hub.docker.com/>

For more examples and ideas, visit:

<https://docs.docker.com/get-started/>

친하게 지내기4~



옆자리 동료와 이야기해보기

Docker가 하는 일을, 적절한 비유로 옆 사람에게 설명해보세요.



2분

FROM



FROM

“ 비유로 말하면

"이 요리는 밥을 베이스로 시작할게요"라고 정하는 것

☰ 정확히 말하면

이미지를 만들 때 어떤 재료(베이스 이미지)로 시작할지 정합니다.
Dockerfile의 항상 첫 줄입니다.

FROM python:3.10-slim

WORKDIR



WORKDIR

“ 비유로 말하면

"오늘 요리는 이 도마 위에서 하실게요"라고 작업 공간을 정하는 것

☰ 정확히 말하면

앞으로의 작업이 이루어질 폴더(작업 디렉토리)를 지정

WORKDIR /app

COPY



COPY

“ 비유로 말하면

내 냉장고에 있는 재료를 도마 위로 옮겨오는 것

☰ 정확히 말하면

내 컴퓨터의 파일을, 만들고 있는 이미지 안으로 복사해 넣습니다.

COPY app.py .



RUN

“ 비유로 말하면

"재료를 씻고 다듬어주세요"처럼, 이미지를 만드는 중간에 필요한 준비 작업

☰ 정확히 말하면

이미지를 만드는 시점에 실행할 명령어
주로 필요한 라이브러리를 설치할 때 씁니다.

RUN pip install flask



EXPOSE

“ 비유로 말하면

"손님은 정문으로 들어오실게요"라고 문 번호를 안내해두는 것

☰ 정확히 말하면

이 컨테이너가 몇 번 문(포트)으로 손님을 받을지 미리 적어두는 안내문

EXPOSE 5000



CMD

“ 비유로 말하면

"자, 이제 완성된 요리를 손님상에 내어드릴게요"라고 마지막 실행 버튼을 누르는 것

☰ 정확히 말하면

컨테이너가 시작될 때 자동으로 실행할 명령어
Dockerfile의 마지막 줄에 주로 씁니다.

CMD ["python", "app.py"]

[실습] 첫 Dockerfile 써보기

🕒 방금 배운 6가지 명령어를 순서대로 조합해 나만의 Dockerfile을 완성합니다.

- ❶ workspace 폴더 안에 Dockerfile 이라는 이름(확장자 없음)의 파일을 만든다
- ❷ FROM python:3.10-slim 을 첫 줄에 적는다
- ❸ WORKDIR /app 을 적는다
- ❹ COPY . . 를 적어 현재 폴더의 모든 파일을 복사한다
- ❺ EXPOSE 5000 과 CMD ["python", "app.py"] 로 마무리한다

[실습] 이미지 빌드해보기

🕒 레시피(Dockerfile)로 실제 봉어빵 틀(이미지)을 만들어봅시다.

- ❶ 터미널에서 Dockerfile이 있는 폴더로 이동한다
- ❷ `docker build -t my-app:1.0 .` 을 입력한다
- ❸ 여러 줄의 진행 메시지가 지나가는 것을 관찰한다 (Dockerfile의 한 줄 = 한 단계)
- ❹ `docker images` 로 방금 만든 이미지가 목록에 있는지 확인한다

✅ `-t my-app:1.0` 은 이미지에 이름표(태그)를 붙이는 옵션입니다.

[실습] 첫 Dockerfile 써보기

```
# 1) 기존 컨테이너 삭제
docker rm -f my-container

# 2) app.py 작성
cat << 'EOF' > app.py
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    message = "Docker yarrrrrrrr!!!!!"
    print(message, flush=True)
    return f"<h1>{message}</h1>"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
EOF
```

[실습] 첫 Dockerfile 써보기

```
# 3) Dockerfile 작성
cat << 'EOF' > Dockerfile
FROM python:3.10-slim
WORKDIR /app
RUN pip install flask
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
EOF

# 4) 캐시 없이 깨끗하게 재빌드
docker build --no-cache -t my-app:1.0 .
```

[실습] 이미지 빌드해보기

```

[→] workplace docker build -t my-app:1.0 .
[+] Building 8.1s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 124B
=> [internal] load metadata for docker.io/library/python:3.10-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/2] FROM docker.io/library/python:3.10-slim@sha256:a78e4529630cfe8c5199cafd6e0c28ee1579a13f86274396d8b6b2d80367aa3a
=> => resolve docker.io/library/python:3.10-slim@sha256:a78e4529630cfe8c5199cafd6e0c28ee1579a13f86274396d8b6b2d80367aa3a
=> => sha256:61cf3235e8849721f7741f4829788e6bbcb1d325448330e69e02cab1bb7c31e 13.87MB / 13.87MB
=> => sha256:1b7200988f192e72703c70486d494e2457935ac9b0f031ac09eb115b01a12d45 30.14MB / 30.14MB
=> => sha256:b114d2ba70e3bfd3fc08940d677acd2d40e63be644e49e260bf191716377f092 1.27MB / 1.27MB
=> => sha256:f670c90339fa2e29e081a3b79c50f6639127085b921d49a18c59b4a7a050522d 248B / 248B
=> => extracting sha256:1b7200988f192e72703c70486d494e2457935ac9b0f031ac09eb115b01a12d45
=> => extracting sha256:b114d2ba70e3bfd3fc08940d677acd2d40e63be644e49e260bf191716377f092
=> => extracting sha256:61cf3235e8849721f7741f4829788e6bbcb1d325448330e69e02cab1bb7c31e
=> => extracting sha256:f670c90339fa2e29e081a3b79c50f6639127085b921d49a18c59b4a7a050522d
=> [2/2] WORKDIR /app COPY . .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:b9900c56b7021c7ebbdd73db3e6801d2a75f8cc04f9ddc71bdbd4cce6e29f3d6
=> => exporting config sha256:b03f4f84149d328d23784bbcd3323621ee6435429a59643f476f69adb7dd3787
=> => exporting attestation manifest sha256:6067c0f58a8c904a7cac7d0f1315f99afb2b5f023a9732c2183e24b2712166b3
=> => exporting manifest list sha256:7488466939cd31b90cb3a899bf2993428e144d8d2fcb21fe01743d2845a56812
=> => naming to docker.io/library/my-app:1.0
=> => unpacking to docker.io/library/my-app:1.0
→ workplace

```


[실습] 이미지 빌드해보기

```

workplace — a02950@SKAX26N00246 — ../workplace

=> => transferring dockerfile: 124B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.10-slim 3.0s
=> [auth] library/python:pull token for registry-1.docker.io      0.0s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                       0.0s
=> [1/2] FROM docker.io/library/python:3.10-slim@sha256:a78e4529630cfe8c5199cafd6e0c28ee1579a13f86274396d8b6b2d80367aa3a 4.9s
=> => resolve docker.io/library/python:3.10-slim@sha256:a78e4529630cfe8c5199cafd6e0c28ee1579a13f86274396d8b6b2d80367aa3a 0.0s
=> => sha256:61cf3235e8849721f7741f4829788e6bbbc1d325448330e69e02cab1bb7c31e 13.87MB / 13.87MB 2.8s
=> => sha256:1b7200988f192e72703c70486d494e2457935ac9b0f031ac09eb115b01a12d45 30.14MB / 30.14MB 4.3s
=> => sha256:b114d2ba70e3bfd3fc08940d677acd2d40e63be644e49e260bf191716377f092 1.27MB / 1.27MB 2.4s
=> => sha256:f670c90339fa2e29e081a3b79c50f6639127085b921d49a18c59b4a7a050522d 248B / 248B 0.4s
=> => extracting sha256:1b7200988f192e72703c70486d494e2457935ac9b0f031ac09eb115b01a12d45 0.4s
=> => extracting sha256:b114d2ba70e3bfd3fc08940d677acd2d40e63be644e49e260bf191716377f092 0.0s
=> => extracting sha256:61cf3235e8849721f7741f4829788e6bbbc1d325448330e69e02cab1bb7c31e 0.2s
=> => extracting sha256:f670c90339fa2e29e081a3b79c50f6639127085b921d49a18c59b4a7a050522d 0.0s
=> [2/2] WORKDIR /app COPY . .                                     0.1s
=> exporting to image                                              0.0s
=> => exporting layers                                              0.0s
=> => exporting manifest sha256:b9900c56b7021c7ebbdd73db3e6801d2a75f8cc04f9ddc71bdbd4cce6e29f3d6 0.0s
=> => exporting config sha256:b03f4f84149d328d23784bbcd3323621ee6435429a59643f476f69adb7dd3787 0.0s
=> => exporting attestation manifest sha256:6067c0f58a8c904a7cac7d0f1315f99afb2b5f023a9732c2183e24b2712166b3 0.0s
=> => exporting manifest list sha256:7488466939cd31b90cb3a899bf2993428e144d8d2fcb21fe01743d2845a56812 0.0s
=> => naming to docker.io/library/my-app:1.0                       0.0s
=> => unpacking to docker.io/library/my-app:1.0                     0.0s

[→ workplace docker images]


```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	5dd0d3e6e255	22.6kB	10.3kB	U
my-app:1.0	7488466939cd	209MB	45.3MB	

```

→ workplace

```

Info →
U In Use

[실습] 컨테이너 실행해보기

🕒 봉어빵 틀(이미지)로 실제 봉어빵(컨테이너)을 구워봅니다.

- ❶ `docker run -d -p 8080:5000 --name my-container my-app:1.0` 을 입력한다
- ❷ `docker ps` 로 컨테이너가 실행 중인지 확인한다
- ❸ 브라우저에서 `http://localhost:8080` 에 접속해본다
- ❹ `docker stop <컨테이너ID>`
`docker rm <컨테이너ID>` 로 정리한다

[실습] 컨테이너 실행해보기

```
[docker build --no-cache -t my-app:1.0 .
my-container
[+] Building 2.5s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 140B
=> [internal] load metadata for docker.io/library/python:3.10-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/python:3.10-slim@sha256:a78e4529630cfe8c5199cafd6e0c28ee1579a13f86274396d8b6b2d80367aa3a
=> => resolve docker.io/library/python:3.10-slim@sha256:a78e4529630cfe8c5199cafd6e0c28ee1579a13f86274396d8b6b2d80367aa3a
=> [internal] load build context
=> => transferring context: 424B
=> CACHED [2/4] WORKDIR /app
=> [3/4] RUN pip install flask
=> [4/4] COPY . .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:64fa59c0f3bc9e42349b3861ed51cafd9961bdd0ecffade9438d3e5b90a70502
=> => exporting config sha256:02103ab17aef7c93bc4d29fee7d30c561c190c9a3c87a1fb175650ec758ae9e0
=> => exporting attestation manifest sha256:a194823e408c43e238792dd06ca45e25041b1c3c56baf21962ec1b4d039cfdef
=> => exporting manifest list sha256:8072ad994f137d573127fd57c6004f88de11d8e712a2f31776cd31dd78720c52
=> => naming to docker.io/library/my-app:1.0
=> => unpacking to docker.io/library/my-app:1.0
[→ workplace docker run -d -p 8080:5000 --name my-container my-app:1.0
5d9ba942d09cb6be282d1647277a341e466dbff40dcdbae34cc4785af6bc2daa
[→ workplace docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
5d9ba942d09c   my-app:1.0 "python app.py"         4 minutes ago Up 4 minutes   0.0.0.0:8080->5000/tcp, [::]:8080->5000/tcp   my-contai
ner
→ workplace
```

-d 옵션



-d 옵션

“ 비유로 말하면

가게 문은 열어두되, 사장님은 뒤에서 조용히 일하는 것과 같습니다.

☰ 정확히 말하면

detached(백그라운드)의 줄임말

터미널을 계속 붙잡고 있지 않고, 뒤에서 조용히 계속 실행되게 합니다.

-p 옵션



-p 옵션

“ 비유로 말하면

건물 정문(내 컴퓨터 포트)과 가게 안쪽 문(컨테이너 포트)을 연결하는 통로

☰ 정확히 말하면

port의 줄임말

-p 5000:5000 은 '내 컴퓨터의 5000번 문을 컨테이너의 5000번 문과 연결하라'는 뜻

--name 옵션



--name 옵션

“ 비유로 말하면

번호표 대신 예쁜 이름표를 달아주는 것

☰ 정확히 말하면

컨테이너에 이름을 붙여줍니다.

이름을 붙이면 복잡한 ID 대신 그 이름으로 컨테이너를 다룰 수 있습니다.

docker run --name my-first-app -d -p 5000:5000 my-app:1.0

--network 옵션



--network 옵션

“ 비유로 말하면

여러 개의 붕어빵 틀(컨테이너)이 각자 다른 건물에 혼자 있으면 서로 못 찾아가요.
하지만 같은 아파트 단지(네트워크) 안에 입주하면, 상대방 동·호수(컨테이너 이름)만 불러도
바로 찾아갈 수 있어요.

☰ 정확히 말하면

docker는 기본적으로 컨테이너마다 서로 다른 방을 줘서,
아무 설정 없이는 컨테이너끼리 이름으로 통신할 수 없습니다.

docker network create board-net

docker run --name board-db --network board-net -d postgres:17

docker run --name board-was --network board-net -e DB_HOST=board-db -d board-was:1.0

-e 옵션



-e 옵션

*-e DB_HOST=board-db는 컨테이너 안에 환경변수를 심어주는 옵션이에요.
"이 앱아, DB는 board-db라는 곳에 있어"라고 쪽지를 남겨주는 것과 같습니다.
앱 코드는 이 쪽지(환경변수)를 읽어서 접속 주소를 결정*

자주 쓰는 Docker 명령어 모음

명령어	내용
docker ps	지금 실행 중인 컨테이너 목록 보기
docker ps -a	멈춘 것까지 포함한 전체 목록 보기
docker exec -it <이름> sh	컨테이너가 남긴 기록(로그) 보기
docker stop <이름>	컨테이너 멈추기
docker rm <이름>	멈춘 컨테이너 삭제

[Quiz]

이미지와 컨테이너의 관계를 가장 잘 설명한 것은?

- 둘은 완전히 같은 것이다
- 이미지는 붕어빵 틀, 컨테이너는 그 틀로 구운 붕어빵이다
- 컨테이너가 이미지보다 먼저 만들어진다
- 이미지는 실행 중, 컨테이너는 저장된 상태다

[Quiz]

이미지와 컨테이너의 관계를 가장 잘 설명한 것은?

- ☐ 둘은 완전히 같은 것이다
- ☒ 이미지는 봉어빵 틀, 컨테이너는 그 틀로 구운 봉어빵이다
- ☐ 컨테이너가 이미지보다 먼저 만들어진다
- ☐ 이미지는 실행 중, 컨테이너는 저장된 상태다
- ☐ 이미지는 설계도(봉어빵 틀), 컨테이너는 그 설계도로 실제 실행 중인 것(봉어빵)

[Quiz]

✓ Bare-metal → VM → Container 순서로 더 가볍고 빨라졌다

✓ Dockerfile은 이미지를 만드는 레시피다

✓ docker build로 이미지를, docker run으로 컨테이너를 만든다

✓ 이미지는 설계도, 컨테이너는 실행되고 있는 것이다

✓ FROM, WORKDIR, COPY, RUN, EXPOSE, CMD 6가지가 핵심이다

✓ -d, -p, --name 옵션으로 실행 방식을 조절

SKALA 3) Cloud – 2. 컨테이너 이해 및 애플리케이션 컨테이너화

3. 네트워크, 스토리지

- 컨테이너를 외부와 연결하고, 데이터를 안전하게 지키는 방법을 배웁니다

네트워크 (Network)



네트워크 (Network)

“ 비유로 말하면

컴퓨터와 컴퓨터를 잇는 도로입니다.

☰ 정확히 말하면

여러 컴퓨터(또는 컨테이너)가 서로 데이터를 주고받을 수 있게 연결된 통로입니다.

포트 (Port)



포트 (Port)

“ 비유로 말하면

아파트의 호수(101호, 102호), 같은 건물(컴퓨터)이라도 몇 호로 찾아가느냐가 다릅니다.

☰ 정확히 말하면

한 컴퓨터 안에서 여러 프로그램을 구분하기 위한 번호
웹사이트는 보통 80번

[실습] 포트 번호 바꿔서 실행해보기

🕒 -p 옵션의 앞뒤 숫자가 바뀌면 무슨 일이 생기는지 직접 확인

- ❶ `docker run -d -p 8080:5000 my-app:1.0` 을 실행한다
- ❷ `http://localhost:8080` 으로 접속해본다 (8080으로 열림에 주목)
- ❸ `http://localhost:5000` 으로도 접속해본다 (이번엔 안 열림)
- ❹ 왜 그런지 옆 사람과 이야기해본다

✅ -p 호스트포트:컨테이너포트 (호스트 쪽 숫자만 내가 접속할 문 번호 지정)

컨테이너끼리는 어떻게 대화할까

 컨테이너는 기본적으로 서로 격리, 같은 '네트워크'에 연결하면 대화

- 같은 네트워크에 있으면, 컨테이너 이름만으로 서로를 찾을 수 있습니다.
- WAS 컨테이너에서 'db' 라는 이름만으로 DB 컨테이너에 연결
- 오늘 종합실습에서 이 방식을 직접 사용해봅니다.

친하게 지내기4~



옆자리 동료와 이야기해보기

포트와 네트워크의 차이를, 다른 비유를 써서 옆 사람에게 설명해보세요.



2분

스토리지가 왜 필요할까 — 기억상실 문제

 *컨테이너를 지우면, 그 안에 저장했던 데이터도 함께 사라집니다.*

- 마치 도시락을 다 먹고 버리면, 안에 있던 음식도 같이 사라지는 것
- 회원가입 데이터처럼 계속 남아있어야 하는 정보에는 치명적인 문제입니다.
- 이 문제를 해결하는 것이 바로 '볼륨'입니다.

볼륨 (Volume)



볼륨 (Volume)

“ 비유로 말하면

도시락과 별개로 마련된 창고. 도시락(컨테이너)을 버려도 창고(볼륨)의 물건은 그대로

☰ 정확히 말하면

컨테이너 바깥에 데이터를 저장해두는 공간
컨테이너가 삭제돼도 볼륨의 데이터는 남습니다.

Bind Mount vs Volume, 쉬운 비교



Bind Mount

- 내 컴퓨터의 특정 폴더를 그대로 연결
- 코드를 수정하면 바로 반영됨
- 개발할 때 특히 편리함



Named Volume

- Docker가 알아서 관리하는 창고
- DB처럼 계속 남아야 할 데이터에 적합
- 어느 컴퓨터로 옮겨도 잘 동작함

[실습] 볼륨 만들어보기

🕒 볼륨을 만들고, 컨테이너에 연결해봅니다.

- ❶ `docker volume create my-data` 로 볼륨을 만든다
- ❷ `docker volume ls` 로 만들어졌는지 확인한다
- ❸ `docker run -d -v my-data:/data alpine sleep 1000` 으로 볼륨을 연결해 실행한다
- ❹ `docker volume inspect my-data` 로 상세 정보를 확인한다

[실습] 볼륨 만들어보기

```
[→ workplace docker volume create my-data
my-data
[→ workplace docker volume ls
DRIVER      VOLUME NAME
local       my-data
[→ workplace docker run -d -v my-data:/data alpine sleep 1000
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
5de55e5ef9c0: Pull complete
df8ce8557afe: Download complete
aa3ec251a2db: Download complete
Digest: sha256:28bd5fe8b56d1bd048e5babf5b10710ebe0bae67db86916198a6eec434943f8b
Status: Downloaded newer image for alpine:latest
3ce4ad99672d5da6a322282cceed4eaab70f153e486f8316380d6ae539dea2e9
[→ workplace docker volume inspect my-data
[
  {
    "CreatedAt": "2026-08-19T05:54:29Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/my-data/_data",
    "Name": "my-data",
    "Options": null,
    "Scope": "local"
  }
]
→ workplace █
```

[Quiz]

컨테이너를 삭제해도 데이터가 남아있게 하려면 무엇을 써야 할까요?

- ☐ 포트(Port)
- ☐ 볼륨(Volume)
- ☐ 이미지(Image)
- ☐ 터미널(Terminal)

[Quiz]

컨테이너를 삭제해도 데이터가 남아있게 하려면 무엇을 써야 할까요?

☐ 포트(Port)

☒ 볼륨(Volume)

☐ 이미지(Image)

☐ 터미널(Terminal)

💡 볼륨은 컨테이너 바깥의 창고이므로, 컨테이너가 사라져도 데이터는 그대로 남습니다.

네트워크 · 스토리지, 이것만은 기억하세요

✓ 포트는 한 컴퓨터 안에서 프로그램을 구분하는 번호

✓ 같은 네트워크의 컨테이너는 이름으로 서로 통신 가능

✓ 볼륨을 쓰면 컨테이너가 사라져도 데이터는 남는다

✓ -p 호스트포트
: 컨테이너포트 로 외부와 연결

✓ 컨테이너를 지우면
안의 데이터도 함께 사라진다

SKALA 3) Cloud – 2. 컨테이너 이해 및 애플리케이션 컨테이너화

4. 앱 이미지, 컨테이너

- 지금까지 배운 것을 모아, 실제 앱 하나를 이미지로 완성

오늘 만드는 앱 소개 — 미니 게시판

우리가 오늘부터 내일까지 함께 만들 앱은, 아주 단순한 3-tier 미니 게시판입니다.



Web (화면)

사용자가 보는 화면
글 목록과 글쓰기 버튼



WAS (서버)

화면의 요청을 받아
처리하는 프로그램



DB (저장소)

게시글 데이터를 실제로
저장하는 곳

앱을 이미지로 만드는 전체 흐름



이미지 태그 (Tag)



이미지 태그 (Tag)

“ 비유로 말하면

붕어빵 틀에 붙이는 이름표 — "팥 붕어빵 v1", "슈크림 붕어빵 v2"

☰ 정확히 말하면

이미지 이름 뒤에 콜론(:)으로 붙이는 버전 이름
태그가 없으면 자동으로 latest가 붙습니다.

***my-app:1.0* 처럼 버전을 명확히 적어두면 나중에 헛갈리지 않습니다.**

이미지 태그 (Tag)

이미지는 여러 층으로 쌓여있다 (살짝 맛보기)

 *Dockerfile의 한 줄, 한 줄이 이미지의 한 '층(레이어)'이 됩니다.*

- 마치 케이크가 여러 층으로 쌓여있는 것과 같습니다.
- 그래서 코드 한 줄만 바뀌어도 이미지 전체를 처음부터 다시 만들 필요 없이, 바뀐 층만 새로 만들면 됩니다.
- 이 원리는 Day2에서 조금 더 자세히 다룹니다.

[실습] 나만의 WAS 이미지 만들기 — 1단계

🕒 미니 게시판의 WAS(서버) 부분을 위한 Dockerfile을 작성합니다.

- ❶ was 폴더를 만들고 이동한다
- ❷ 앞에서 코드(app.py)를 폴더 안에 넣는다 (또는 간단히 직접 작성)
- ❸ Dockerfile을 작성한다 (FROM, WORKDIR, COPY, RUN, EXPOSE, CMD)
- ❹ 작성한 내용을 옆 사람과 비교해본다

[실습] 나만의 WAS 이미지 만들기 — 2단계

🕒 작성한 **Dockerfile**로 실제 이미지를 빌드합니다.

- ❶ `docker build -t my-was:1.0 .` 을 실행한다
- ❷ 진행 로그를 보며 Dockerfile의 어느 줄이 지금 실행되고 있는지 따라가본다
- ❸ `docker images` 로 이미지가 만들어졌는지 확인한다
- ❹ 에러가 나면 당황하지 말고 메시지를 천천히 읽어본다

[실습] 컨테이너로 띄우기

🕒 만든 이미지를 실제로 실행해봅니다.

- ❶ `docker run -d --name my-was -p 8080:5000 my-was:1.0` 을 실행한다
- ❷ `docker ps` 로 정상 실행 중인지 확인한다
- ❸ `docker logs my-was` 로 시작 로그를 확인한다

[실습] 브라우저로 확인하기

🕒 실제로 눈으로 결과 확인

- ❶ 브라우저를 열고 `http://localhost:8080` 으로 접속한다
- ❷ 화면이 정상적으로 보이는지 확인한다
- ❸ 안 보인다면 `docker ps`, `docker logs` 순서로 원인을 점검한다

이미지 이름 예시

이미지	역할
hello-world	Docker 설치를 확인하는 테스트용
python:3.10-slim	우리가 만든 WAS (서버) 이미지 베이스
nginx:alpine	화면을 보여주는 가벼운 웹서버
postgreSQL:17.0	데이터를 저장하는 공식 DB 이미지
my-was:1.0 / my-app:1.0	직접 빌드한 이미지

자주 발생하는 실수 모음

증상	흔한 원인
build 시 오류	Dockerfile 파일명 오타, 대소문자 확인
이미지는 있는데 실행 안 됨	docker run 명령의 이미지 이름, 태그 오타 확인
브라우저에 안 보임	-p 옵션의 포트 번호가 EXPOSE와 맞는지 확인
바로 꺼짐	docker logs로 에러 메시지 확인



옆자리 동료와 이야기해보기

오늘 배운 것 중 가장 헷갈렸던 부분 하나를 옆 사람에게 말해보고
서로 답을 찾아보세요.



2분

[Quiz]

Dockerfile에서 이미지 실행 시 자동으로 실행할 명령어를 적는 곳은?

☐ FROM

☐ COPY

☐ CMD

☐ EXPOSE

[Quiz]

Dockerfile에서 이미지 실행 시 자동으로 실행할 명령어를 적는 곳은?

☐ FROM

☐ COPY

☒ **CMD**

☐ EXPOSE



CMD는 컨테이너가 시작될 때 자동으로 실행되는 명령어를 정의합니다.

Day 1 이론, 전체 되돌아보기

✓ 터미널로 컴퓨터와 대화하는 법

✓ Dockerfile 6가지 핵심 명령어

✓ 직접 이미지를 만들고 컨테이너로 실행

✓ 컨테이너를 비유로 이해
(도시락, 붕어빵 틀)

✓ 포트와 볼륨으로 연결과 저장을 이해

✓ 이제 종합실습에서 이 모든 걸 하나로

SKALA 3) Cloud – 2. 컨테이너 이해 및 애플리케이션 컨테이너화

5. 종합 실습 1

- 오늘 배운 모든 것을 모아, **Web-WAS-DB 3-tier** 미니 게시판을 완성

시작 전, 실습 환경 재확인

종합실습을 시작하기 전, 아래 항목을 다시 한번 확인해주세요.

- ☐ docker --version 이 정상적으로 출력되는가
- ☐ 지금까지 실습에서 켜둔 컨테이너를 docker ps -a 로 확인했는가
- ☐ 필요 없는 컨테이너는 docker rm 으로 정리했는가
- ☐ 텍스트 에디터가 실습 폴더를 열고 있는가

코드 리뷰란 무엇인가



코드 리뷰는 — 서로의 코드를 보며 함께 이야기하는 것

- "이 부분은 왜 이렇게 작성했어요?" 라고 물어보는 것부터 시작합니다.
- 정답을 맞히는 시험이 아니라, 서로 다른 접근을 보고 배우는 시간입니다.
- 실무에서도 코드 리뷰는 실력을 가장 빠르게 키우는 방법 중 하나입니다.

오늘의 진행 방식



옆자리 동료와 함께

두 명이 서로 동료가 되어,
각자 작성하고 서로의 코드를 리뷰



체크리스트로

정답을 바로 알려주는 대신,
체크리스트로 스스로 점검

종합실습 1 목표

🎯 *오늘 배운 **Linux**, 컨테이너 기본, 네트워크, 스토리지를 하나로*

- Web, WAS, DB 세 개의 컨테이너를 각각 만들고, 서로 연결해 하나의 서비스로 완성
- **Day 2**에서 배울 **Compose** 없이, **docker run**과 네트워크 연결만으로 완성

3-tier 구조, 다시 짚고 가기

세 부분이 각자의 역할을 맡아 하나의 서비스를 완성



Web

- 정적 화면(HTML) 역할
- nginx로 서비스



WAS

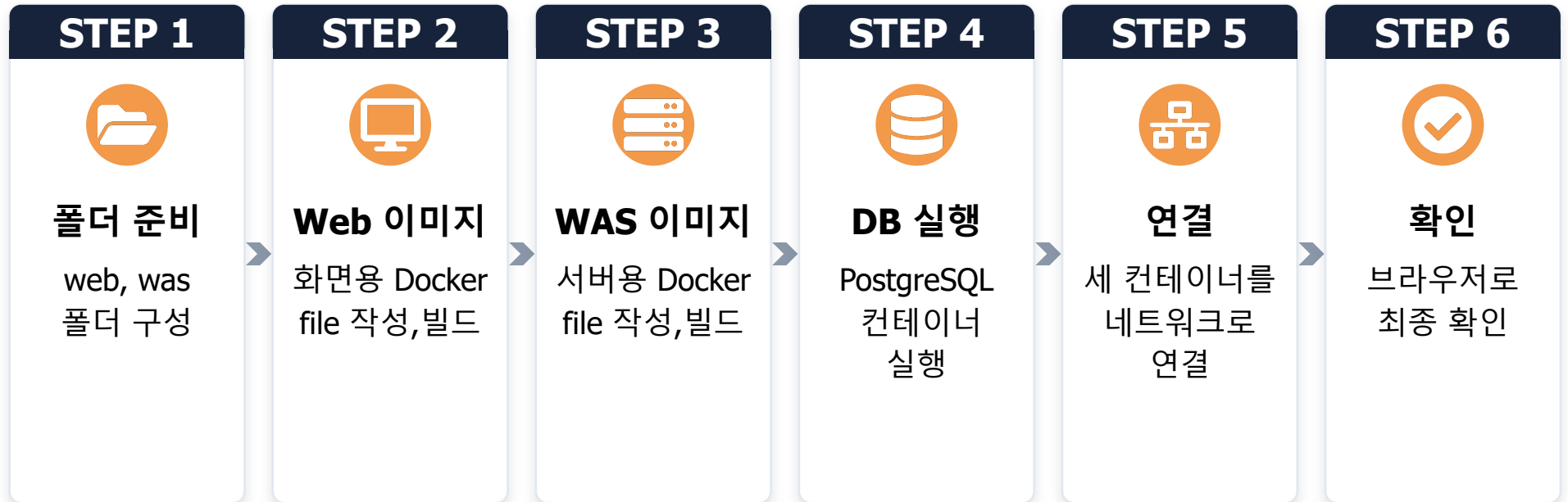
- 요청 처리하는 프로그램
- Web과 DB 사이 연결



DB

- 게시글 데이터를 실제로 저장
- PostgreSQL을 사용

진행 순서 로드맵



[Step 0] 실습 환경 정리하기

🕒 이전 실습에서 쓰던 컨테이너를 정리하고 새 실습을 시작합니다.

- ❶ `docker ps -a` 로 남아있는 컨테이너를 확인한다.
- ❷ `docker rm -f my-container` 로 이전 컨테이너를 삭제한다 (포트 충돌 방지)
- ❸ `docker ps -a` 로 깨끗하게 정리됐는지 다시 확인한다

[Step 1] 프로젝트 폴더 준비하기

🕒 **Linux 명령어로 오늘 실습에 쓸 폴더 구조를 만듭니다.**

- ❶ `mkdir board && cd board` 로 프로젝트 폴더를 만든다
- ❷ `mkdir web was` 로 두 개의 하위 폴더를 만든다
- ❸ 각 폴더에 간단한 코드를 직접 작성한다
- ❹ `ls -al` 로 구조를 확인한다

[Step 2] Web(화면) Dockerfile 작성

정적 화면을 보여주는 아주 단순한 Dockerfile입니다.

```
# web/Dockerfile
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

💡 nginx는 화면을 보여주는 데 특화된 가벼운 웹서버 이미지입니다.

[Step 3] WAS(서버) Dockerfile 작성

연습한 것과 유사한 구조입니다.

```
# was/Dockerfile
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

💡 **requirements.txt** 안에는 **flask, psycpg2-binary** 두 줄이면 충분

[Step 4] DB 컨테이너 실행하기

🎯 공식 PostgreSQL 이미지를 이용해 DB 컨테이너를 실행

- ❶ `docker network create board-net` 으로 전용 네트워크를 만든다
- ❷ `docker run --name board-db --network board-net -e POSTGRES_PASSWORD=mypassword -e POSTGRES_DB=board -d postgres:17` 실행
- ❸ `docker ps` 로 DB가 정상 기동됐는지 확인한다

[Step 5] 세 컨테이너를 네트워크로 연결하기

🎯 **WAS와 Web도 같은 네트워크에 연결해 하나의 서비스로 완성합니다.**

- ❶ `docker build -t board-was:1.0 ./was` 로 WAS 이미지를 빌드한다
- ❷ `docker run --name board-was --network board-net -e DB_HOST=board-db -p 5001:5000 -d board-was:1.0` 을 실행한다
- ❸ `docker build -t board-web:1.0 ./web` 로 Web 이미지를 빌드한다
- ❹ `docker run --name board-web --network board-net -p 8080:80 -d board-web:1.0` 실행

✅ **DB_HOST=board-db 컨테이너 이름 그대로 주소로 쓸 수 있는 것이 네트워크**

[Step 6] 브라우저로 실행 확인하기

🕒 세 컨테이너가 함께 잘 동작하는지 최종 확인

- ❶ `docker ps` 로 세 컨테이너가 모두 Up 상태인지 확인한다
- ❷ 브라우저에서 `http://localhost:8080` (Web)에 접속한다
- ❸ 화면에서 글쓰기를 시도해, WAS를 거쳐 DB에 저장되는지 확인한다
- ❹ `docker logs board-was` 로 DB 연결 로그를 확인한다

예시화면



→ board mkdir board && cd board

→ board mkdir web was

→ board ls -al

total 0

drwxr-xr-x 4 user staff 128 8월 19 15:02 .

drwxr-xr-x 3 user staff 96 8월 19 15:02 ..

drwxr-xr-x 2 user staff 64 8월 19 15:02 was

drwxr-xr-x 2 user staff 64 8월 19 15:02 web

→ board cat web/Dockerfile

web/Dockerfile

FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

예시화면

```
→ board cat was/Dockerfile
# was/Dockerfile
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]

→ board docker network create board-net
d3f8a1e29b7c4a6e9f0123456789abcdef0123456789abcdef0123456789ab

→ board docker run --name board-db --network board-net \
-e POSTGRES_PASSWORD=mypassword -e POSTGRES_DB=board -d postgres:17
7a2c9e4f1b83d5e6a7c8b9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0
```


예시화면



→ board docker ps

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS	NAMES
7a2c9e4f1b83	postgres:17	"docker-entrypoint.s..."	Up 5 seconds	5432/tcp	board-db

→ board docker build -t board-was:1.0 ./was

[+] Building 3.2s (10/10) FINISHED

=> [1/4] FROM docker.io/library/python:3.10-slim

=> [2/4] WORKDIR /app

=> [3/4] COPY requirements.txt .

=> [4/4] RUN pip install --no-cache-dir -r requirements.txt

=> exporting to image

=> => naming to docker.io/library/board-was:1.0

→ board docker run --name board-was --network board-net \

-e DB_HOST=board-db -p 5001:5000 -d board-was:1.0

b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2

예시화면

→ board docker build -t board-web:1.0 ./web

[+] Building 1.5s (8/8) FINISHED

=> [1/3] FROM docker.io/library/nginx:alpine

=> [2/3] COPY index.html /usr/share/nginx/html/index.html

=> exporting to image

=> => naming to docker.io/library/board-web:1.0

→ board docker run --name board-web --network board-net \

-p 8080:80 -d board-web:1.0

c9d8e7f6a5b4c3d2e1f0a9b8c7d6e5f4a3b2c1d0e9f8a7b6c5d4e3f2a1b0c9d8

→ board docker ps

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS	NAMES
c9d8e7f6a5b4	board-web:1.0	"/docker-entrypoint..."	Up 3 seconds	0.0.0.0:8080->80/tcp	board-web
b1c2d3e4f5a6	board-was:1.0	"python app.py"	Up 8 seconds	0.0.0.0:5001->5000/tcp	board-was
7a2c9e4f1b83	postgres:17	"docker-entrypoint.s..."	Up 20 seconds	5432/tcp	board-db

예시화면



```
→ board docker logs board-was
* Serving Flask app 'app'
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.18.0.3:5000
[DB 연결 성공] board-db:5432 접속 완료
```

코드 리뷰 체크리스트

옆자리 동료의 코드를 볼 때 아래 항목을 함께 확인해보세요.

- ☐ Dockerfile의 FROM에 적절한 베이스 이미지를 사용했는가
- ☐ 불필요한 파일까지 COPY하지는 않았는가
- ☐ EXPOSE 포트와 실제 -p 옵션의 포트가 맞는가
- ☐ 환경 변수(DB_HOST 등) 이름이 코드와 일치하는가
- ☐ 컨테이너 이름이 서로 겹치지 않는가

친하게 지내기6(?)~



옆자리 동료와 이야기해보기

옆자리 동료와 자리를 바꿔 앉아,
서로의 **Dockerfile**을 체크리스트로 리뷰해보세요.



10분

제출물

소스 파일

- web/Dockerfile, was/Dockerfile (+ was 소스코드, requirements.txt)

실행 증빙 자료

- 실행결과 화면 Capture
 - docker ps 결과 (컨테이너 3개)
 - + 브라우저 화면
- 동료와의 코드 리뷰 내용 정리

제출 기한 및 제출 방법

- 캠퍼스_반_이름_종합실습명.zip으로 제출 (판교캠퍼스_5반_홍길동_day1종합실습.zip)
 - 동료와의 코드리뷰 내용을 정리하여 PDF 파일로 생성
 - 소스 파일 + PDF 파일 구성으로 압축해서 제출
- 기한 : 과정 종료 시까지 (~21:00 제출 접수, 제출 기한 초과 시 감점 처리 예정)

자주 나오는 질문 모음

질문	답
왜 DB_HOST에 localhost를 쓰면 안 되나요?	• localhost는 '컨테이너 자기 자신'을 뜻하기 때문 • 다른 컨테이너를 가리키려면 그 컨테이너의 이름을 사용
포트 번호는 아무거나 써도 되나요?	• 가능 • 다만 이미 다른 프로그램이 쓰고 있는 번호와 겹치면 에러
컨테이너를 지우면 이미지도 지워지나요?	• 불가 • 이미지(틀)와 컨테이너(붕어빵)는 별개 • docker rmi로 따로 지워야 함

트러블슈팅 가이드

증상	점검 순서
WAS가 DB에 연결 못 함	① 같은 --network 인지 ② DB_HOST 값이 컨테이너 이름과 같은지
Web에서 화면이 안 보임	① docker ps 로 board-web 이 Up 인지 ② -p 포트 확인
글쓰기가 저장되지 않음	docker logs board-was 로 에러 메시지 확인

종합 실습 1 셀프 점검

아래 항목에 모두 체크할 수 있다면 오늘 실습을 완주한 것입니다.

- ☐ Web, WAS 각각의 Dockerfile을 직접 작성했다
- ☐ DB 컨테이너를 실행하고 네트워크에 연결했다
- ☐ 세 컨테이너가 이름으로 서로 통신하는 것을 확인했다
- ☐ 브라우저에서 글쓰기 → 저장까지 전체 흐름을 확인했다
- ☐ 동료와 서로의 코드를 리뷰했다

종합 실습 1 정리

- ✓ Web, WAS, DB 세 개의 컨테이너 각각 생성
- ✓ docker network로 세 컨테이너를 연결했다
- ✓ 환경 변수로 DB 접속 정보를 컨테이너에 전달
- ✓ 코드 리뷰를 통해 서로 다른 접근 방식을 배웠다
- ✓ 내일은 이 과정을 Docker Compose로 훨씬 간단하게 만들어봅니다

Day 1 마무리 이야기

오늘 배운 내용을 팀 단위로 모여 이야기 나눠보세요.

오늘 가장 어려웠던 개념 하나를 자유롭게 이야기해봅시다.

이미지와 컨테이너의 관계를 자신만의 비유로 다시 설명해봅시다.

3-tier 실습에서 세 컨테이너를 연결할 때 어떤 점이 헷갈렸나요?

내일 **Compose**를 배우면 오늘 실습이 어떻게 더 쉬워질지 예상해봅시다.

컨테이너 이해 및 애플리케이션 컨테이너화

- 
- 6. 1일차 리뷰
 - 7. 컨테이너 구조
 - 8. Docker Compose
 - 9. 종합 실습 (Compose로 완성)

SKALA 3) Cloud – 2. 컨테이너 이해 및 애플리케이션 컨테이너화

6. 1일차 리뷰

- 본격적으로 시작하기 전, 어제 배운 내용을 가볍게 복습합니다.

어제 배운 것, 한눈에 다시 보기

✓ **터미널로 컴퓨터와 대화하는 법**
(pwd, ls, cd, mkdir 등)

✓ **Dockerfile 6가지 명령어**
(FROM, WORKDIR, COPY, RUN, EXPOSE, CMD)

✓ **Web-WAS-DB, 3-tier 구조로**
미니 게시판 완성

✓ 컨테이너 = 도시락, 이미지 = 봉어빵 틀

✓ 포트 연결, 볼륨으로 데이터를 지킨다

[Quiz]

컨테이너 안에 저장한 데이터가 삭제되지 않게 하려면?

- 포트 번호를 바꾼다
- 이미지 태그를 latest로 한다
- 볼륨을 연결한다
- 컨테이너 이름을 길게 짓는다

[Quiz]

컨테이너 안에 저장한 데이터가 삭제되지 않게 하려면?

- 포트 번호를 바꾼다
- 이미지 태그를 latest로 한다
- ✓ 볼륨을 연결한다
- 컨테이너 이름을 길게 짓는다

💡 볼륨은 컨테이너 바깥의 창고이므로, 컨테이너가 삭제돼도 데이터가 남습니다.

[Quiz]

다른 컨테이너의 이름으로 서로 통신하려면 무엇이 같아야 할까요?

- ☐ 이미지 태그
- ☐ 네트워크(--network)
- ☐ 포트 번호
- ☐ 볼륨 이름

[Quiz]

다른 컨테이너의 이름으로 서로 통신하려면 무엇이 같아야 할까요?

☐ 이미지 태그

☒ 네트워크(--network)

☐ 포트 번호

☐ 볼륨 이름

💡 같은 네트워크에 연결된 컨테이너끼리 이름으로 서로를 찾을 수 있습니다.

헛갈리기 쉬운 개념 다시 — 이미지 vs 컨테이너



이미지 (Image)

- 붕어빵 '틀'
- 저장되어 있는 상태
- 하나로 여러 컨테이너 생성 가능



컨테이너 (Container)

- 실제로 구워진 '붕어빵'
- 실행되고 있는 상태
- 같은 이미지로 여러 개 생성 가능

헛갈리기 쉬운 개념 다시 — Bind Mount vs Volume



Bind Mount

- 내 컴퓨터 폴더를 직접 연결
- 개발 중 실시간 반영에 유용



Named Volume

- Docker가 알아서 관리
- DB 등 영속 데이터에 적합

오늘 배울 것 미리보기

어제 손으로 직접 했던 연결 작업을, 오늘은 도구의 힘으로

STEP 1



컨테이너 구조

컨테이너의
진짜 정체를
더 깊이 이해

STEP 2



Registry

이미지를 창고에
보관하는 법

STEP 3



Docker Compose

여러 컨테이너를
한 번에 지휘

STEP 4



종합실습 2

어제 실습을
Compose로 재완성

SKALA 3) Cloud – 2. 컨테이너 이해 및 애플리케이션 컨테이너화

7. 컨테이너 구조

- 컨테이너가 사실은 무엇으로 이루어져 있는지, 조금 더 깊이 들어갑니다.

컨테이너는 사실 무엇일까 — 프로세스 이야기



컨테이너는 사실 특별한 종류의 프로세스입니다.

- 컨테이너는 가상의 컴퓨터가 아니라,
우리 컴퓨터(호스트) 안에서 살짝 '눈가림'된 프로세스입니다.
- 그래서 컨테이너는 VM처럼 무겁지 않고, 순식간에 실행될 수 있는 것입니다.

Namespace



Namespace

“ 비유로 말하면

칸막이가 있는 독서실. 같은 방에 있어도 서로 다른 사람은 안 보이게 시야를 막아둔 것입니다.

☰ 정확히 말하면

컨테이너마다 프로세스, 네트워크 등을 서로 안 보이게 나눠주는 리눅스 기술이 덕분에 컨테이너는 서로 독립된 것처럼 보입니다.

cgroups



cgroups

“ 비유로 말하면

각자에게 정해진 용돈만큼만 쓸 수 있게 하는 것

☰ 정확히 말하면

컨테이너마다 CPU, 메모리를 얼마나 쓸 수 있는지 제한하는 리눅스 기술.
한 컨테이너가 자원을 독차지하지 못하게 막아줍니다.

[실습] 컨테이너도 결국 프로세스

직접 눈으로 확인하면 이해가 훨씬 빨라집니다.



```
$ docker run -d --name demo nginx
```

컨테이너 안에서 확인 (Namespace 덕분에 자기 혼자만 보임)

```
$ docker exec demo cat /proc/1/status | head -2
```

Name: nginx

State: S (sleeping)

→ 컨테이너 안에서는 PID 1이 nginx 자신! (호스트에서는 PID 1이 아님)

호스트(내 컴퓨터)에서 컨테이너의 실제 프로세스를 확인

```
$ docker top demo
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	4821	4802	0	15:03	?	00:00:00	nginx: master process
101	4867	4821	0	15:03	?	00:00:00	nginx: worker process

→ 호스트 관점에서는 평범한 프로세스 번호(PID)를 가지고 있음

→ 즉, 컨테이너 안(PID 1)과 호스트에서 보는 번호(PID 4821)가 서로 다름 = Namespace가 "번호표"를 새로 매겨준 것

컨테이너 네트워크, 다시 한번

 어제 우리가 만든 *board-net*도 사실 *Docker*가 만들어준 '작은 도로망'

- Docker는 기본적으로 컨테이너를 위한 가상 도로(bridge network)를 자동으로 준비
- 같은 도로 위에 있는 컨테이너끼리는 이름을 주소처럼 사용

컨테이너 플랫폼이란 무엇인가

 ***Docker는 컨테이너 기술 그 자체 보다는, 쉽게 쓰도록 도와주는 '도구 모음'***

- 실제로 컨테이너를 실행하는 더 낮은 수준의 엔진들이 Docker 내부에서 동작 중
- 지금은 " **Docker** 말고도 다른 도구가 있구나 " 정도만 알아도 충분

Docker 말고 다른 것도 있다?

실무에서 가끔 마주칠 수 있는 이름들을 가볍게 소개합니다.

이름	설명
Docker	- 가장 널리 쓰이는 표준 도구 - 배우는 중
containerd	Docker 내부에서도 실제로 일하는 엔진
Podman	Docker와 비슷하지만 관리자 프로그램 없이 동작
CRI-O	쿠버네티스 전용으로 만들어진 경량 엔진

친해지길 바라~



옆자리 동료와 이야기해보기

컨테이너가 **VM**보다 가벼운 이유를, **Namespace**라는 단어를 써서 설명해보세요.



2분

[Quiz]

컨테이너마다 **CPU·메모리** 사용량을 제한하는 기술은?

☐ Namespace

☐ cgroups

☐ Dockerfile

☐ Registry

[Quiz]

컨테이너마다 **CPU·메모리 사용량을 제한하는 기술은?**

☐ Namespace

☒ **cgroups**

☐ Dockerfile

☐ Registry

💡 **cgroups**는 자원 사용량에 상한선을 두는 기술.
Namespace는 '시야'를, **cgroups**는 '용돈'을 나눠준다고 기억하세요.

컨테이너 구조, 이것만은 기억하세요

✓ 컨테이너는 가상 컴퓨터가 아니라 특별한 프로세스

✓ cgroups는 자원 사용량을 제한

✓ Namespace는 서로 안 보이게 나눠주는 칸막이

✓ Docker 외에도 containerd, Podman 같은 도구

SKALA 3) Cloud – 2. 컨테이너 이해 및 애플리케이션 컨테이너화

8. Docker Compose

- 이미지를 창고에 보관하고, 여러 컨테이너를 한 번에 지휘하는 법

Registry



Registry

“ 비유로 말하면

이미지를 보관하는 창고

내가 만든 붕어빵 틀을 다른 사람도 꺼내 쓸 수 있게 맡겨두는 곳입니다.

☰ 정확히 말하면

만든 이미지를 저장하고 공유할 수 있는 저장소

다른 컴퓨터에서도 이 창고에서 이미지를 꺼내 쓸 수 있습니다.

Docker Hub 소개



가장 유명한 공개 창고, *Docker Hub*입니다.

- 어제 사용한 nginx, postgres, python 이미지도 모두 이 Docker Hub에서 가져온 것
- 회사에서는 외부에 공개하고 싶지 않은 이미지를 위해, 자체 비공개 창고(**Private Registry**)를 따로 두기도 합니다.

[실습] 이미지에 태그 붙이기

🕒 **Registry에 올리기 전, 이미지에 창고 주소가 담긴 이름표를 붙입니다.**

- ❶ `docker tag board-was:1.0 <Docker Hub 아이디>/board-was:1.0` 처럼 태그를 붙인다
- ❷ `docker images` 로 새 태그가 붙은 것을 확인한다
- ❸ 태그는 '이 이미지가 어느 창고, 어느 칸에 들어갈지'를 정하는 이름표입니다

<Docker Hub 아이디> 자리에는 본인이 회원가입한 실제 Docker Hub 아이디를 넣어야 다음 단계(push)에서 정상 업로드됩니다.

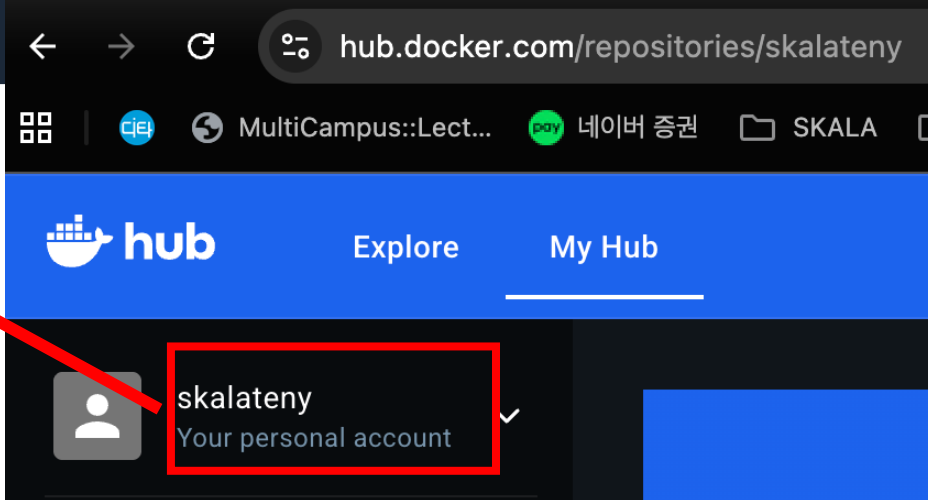
[실습] 이미지에 태그 붙이기

직접 눈으로 확인하면 이해가 훨씬 빨라집니다.

```
$ docker tag board-was:1.0 skalateny/board-was:1.0
```

```
$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
skalateny/board-was	1.0	a1b2c3d4e5f6	10 minutes ago	180MB
board-was	1.0	a1b2c3d4e5f6	10 minutes ago	180MB
board-web	1.0	f6e5d4c3b2a1	12 minutes ago	45MB



이미지 올리기 (Push) 개념



***docker push* 명령으로 내 이미지를 창고에 올립니다.**

- **docker login → docker tag → docker push**, 이 순서를 기억하세요.
- 로그인 없이는 창고 문이 열리지 않습니다.

이미지 받기 (Pull) 개념



***docker pull** 명령으로 참고에서 이미지를 꺼내옵니다.*

- 다른 컴퓨터에서도 같은 이미지를 pull 받으면, 완전히 똑같은 프로그램을 실행할 수 있습니다.
- 사실 우리가 어제 썼던 **docker run postgres:17** 도, 처음 실행할 때 자동으로 pull이 먼저 일어난 것입니다.

여러 컨테이너를 한 번에? — 문제 제기

 어제 우리는 *Web, WAS, DB*를 켤 때 *docker run*을 세 번이나 따로 입력

- 컨테이너가 점점 늘어나면, 매번 순서대로 옵션을 다 기억해서 입력하기 너무 번거롭습니다.
- 이 문제를 해결하는 도구가 바로 오늘 배울 Docker Compose

Docker Compose



Docker Compose

“ 비유로 말하면

오케스트라 지휘자로 여러 연주자(컨테이너)에게 각자의 악보를 미리 나눠주고, 지휘봉 한 번으로 다 함께 연주를 시작하게 합니다.

☰ 정확히 말하면

여러 컨테이너를 하나의 파일에 미리 적어두고, 명령어 한 번으로 한꺼번에 실행·종료할 수 있게 해주는 도구입니다.

Docker Compose

Compose가 해주는 일 세 가지

- ① 여러 컨테이너를 파일 하나로 정리해서 관리
- ② 컨테이너 간 네트워크를 자동으로 만들어줌 (**docker network create** 안 해도 됨)
- ③ **up** 명령어 하나로 전부 켜고, **down** 명령어 하나로 전부 끄

docker-compose.yml, 첫인상

*낯설어 보이지만, 사실 어제 쓴 **docker run** 옵션들을 파일로 옮겨 적은 것뿐입니다.*

```
services:
  was:
    build: ./was
    ports:
      - "5000:5000"
    environment:
      - DB_HOST=db
  db:
    image: postgres:17
    environment:
      - POSTGRES_PASSWORD=1234
```

💡 낯선 글자가 많아 보여도, 하나씩 뜯어보면 어제 배운 옵션들과 똑같습니다.



services

“ 비유로 말하면

지휘자의 악보에서, "오늘 연주할 사람들" 목록입니다.

☰ 정확히 말하면

이 Compose 파일로 실행할 컨테이너들을 적는 가장 큰 영역입니다.

ports



ports

“ 비유로 말하면

어제 배운 -p 옵션과 똑같습니다.

☰ 정확히 말하면

호스트 포트와 컨테이너 포트를 연결합니다. "5000:5000" 처럼 따옴표로 감싸 적습니다.

environment



environment

“ 비유로 말하면

어제 배운 `-e` 옵션과 똑같습니다.

☰ 정확히 말하면

컨테이너 안에서 사용할 환경 변수를 적습니다. DB 주소나 비밀번호 등을 여기 적습니다.

depends_on



depends_on

“ 비유로 말하면

"이 요리는 저 요리가 끝난 다음에 시작하세요"라고 순서를 정하는 것입니다.

☰ 정확히 말하면

이 서비스가 실행되기 전에, 먼저 실행되어야 할 다른 서비스를 지정합니다.

volumes (Compose)



volumes (Compose)

“ 비유로 말하면

어제 배운 -v 옵션과 똑같습니다.

☰ 정확히 말하면

데이터를 보존할 볼륨을 연결합니다. 파일 맨 아래에 볼륨 이름을 한 번 더 선언해야 합니다.

Compose 명령어 — 켜고 끄기

명령어	의미
docker compose up -d	파일에 적힌 모든 컨테이너를 한 번에 실행
docker compose down	모든 컨테이너를 한 번에 종료·삭제

build vs image



build vs image

“ 비유로 말하면

build는 "레시피 보고 직접 만들기", image는 "이미 만들어진 것 사오기"입니다.

☰ 정확히 말하면

build는 Dockerfile로 직접 이미지를 만들 때, image는 이미 존재하는 이미지를 그대로 쓸 때 사용.

우리가 만든 web·was는 build를, 공식 제공되는 db는 image를 사용합니다.

Compose 파일, 자주 하는 실수

실수	고치는 법
들여쓰기에 탭 사용	스페이스만 사용하도록 통일
콜론(:) 뒤에 공백 빠뜨림	key: value 처럼 콜론 뒤 한 칸 띄우기
따옴표 짝이 안 맞음	"8080:80" 처럼 여는 따옴표와 닫는 따옴표 확인

Compose 명령어 — 확인하기

명령어	의미
docker compose ps	• 지금 실행 중인 서비스 목록 확인
docker compose logs -f <서비스명>	• 특정 서비스의 로그를 실시간으로 확인
docker compose build	• Dockerfile을 다시 읽어 이미지 재생성

1일차 방식 vs 2일차 방식

1일차 — docker run

- 명령어를 여러 번 따로 입력
- 네트워크를 직접 만들어야 함
- 컨테이너가 늘어나면 관리가 복잡해짐

2일차 — Docker Compose

- 파일 하나에 전부 선언
- 네트워크가 자동으로 생성됨
- up/down 명령 하나로 전체 관리

[실습] 첫 Compose 파일 써보기

🕒 가장 단순한 형태의 **docker-compose.yml**을 직접 작성합니다.

- ❶ 새로 폴더를 만들고 그 폴더에 docker-compose.yml 파일을 만든다
- ❷ services 아래에 web 이라는 이름으로 image: nginx 를 적는다
- ❸ ports에 "8090:80" 을 추가한다
- ❹ 들여쓰기(공백)가 맞는지 다시 한번 확인한다

✅ **YAML** 파일은 들여쓰기가 아주 중요합니다 — 탭 대신 스페이스를 사용하세요.

[실습] Compose로 실행해보기

🕒 작성한 파일로 실제 컨테이너를 실행합니다.

- ❶ `docker compose up -d` 를 실행한다
- ❷ `docker compose ps` 로 정상 실행됐는지 확인한다
- ❸ 브라우저에서 `http://localhost:8090` 에 접속해본다
- ❹ `docker compose down` 으로 깔끔하게 정리한다

***docker compose ps의 출력에서 컨테이너 이름 앞에 폴더명이 자동으로 붙어요
(폴더명-web-1 형태)***

[실습] Compose로 실행해보기 (예상화면)

다들 수도 있지만 참고로 보세요

```
$ mkdir compose-practice && cd compose-practice
```

```
$ cat << 'EOF' > docker-compose.yml
```

```
services:
```

```
  web:
```

```
    image: nginx
```

```
    ports:
```

```
      - "8090:80"
```

```
EOF
```

```
$ cat docker-compose.yml
```

```
services:
```

```
  web:
```

```
    image: nginx
```

```
    ports:
```

```
      - "8090:80"
```

[실습] Compose로 실행해보기 (예상화면)

다들 수도 있지만 참고로 보세요

```
$ docker compose up -d
```

```
[+] Running 2/2
```

```
✓ Network compose-practice_default Created 0.1s
```

```
✓ Container compose-practice-web-1 Started 0.4s
```

```
$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
compose-practice-web-1	nginx	"/docker-entrypoint...."	web	5 seconds ago	Up 4 seconds	0.0.0.0:8090->80/tcp

브라우저에서 **http://localhost:8090** 접속 시
nginx 기본 환영 페이지("Welcome to nginx!")가 뜨면 성공입니다.

[정리] Compose로 실행해보기 (예상화면)

다들 수도 있지만 참고로 보세요

```
$ docker compose down
[+] Running 2/2
✓ Container compose-practice-web-1   Removed      0.3s
✓ Network compose-practice_default   Removed      0.1s
```

docker compose down을 실행하면 컨테이너뿐 아니라
docker compose up이 자동으로 만든 네트워크(**compose-practice_default**)까지 정리

"Compose가 네트워크까지 알아서 만들고 치워준다"

친해지길 바래2~



옆자리 동료와 이야기해보기

Compose 파일에서 **depends_on**이 하는 역할을 옆 사람에게 설명해보세요.



2분

[Quiz]

여러 컨테이너를 한 번에 실행하는 **Compose** 명령어는?

- ☐ docker build
- ☐ docker compose up -d
- ☐ docker ps
- ☐ docker rm

[Quiz]

여러 컨테이너를 한 번에 실행하는 **Compose** 명령어는?

☐ docker build

☒ **docker compose up -d**

☐ docker ps

☐ docker rm

 **docker compose up -d** 는 파일에 적힌 모든 서비스를 한 번에 백그라운드로 실행

Docker Compose, 이것만은 기억하세요

✓ Registry는 이미지를 보관하는 창고

✓ Compose는 여러 컨테이너를 한 파일로 관리하는 지휘자

✓ up/down으로 전체를 켜고 끌 수 있다

✓ tag → push → pull 순서로 이미지를 공유

✓ services, ports, environment, depends_on, volumes가 핵심 문법

SKALA 3) Cloud – 2. 컨테이너 이해 및 애플리케이션 컨테이너화

9. 종합 실습 2

- 손으로 직접 연결했던 3-tier 미니 게시판을 **Compose**로 훨씬 쉽게 완성

시작 전, 실습 환경 재확인

종합실습을 시작하기 전, 아래 항목을 다시 한번 확인해주세요.

- ☐ 어제 만든 board 폴더가 그대로 남아있는가
- ☐ docker ps -a 로 켜져 있는 이전 컨테이너를 정리했는가
- ☐ docker compose version 이 정상적으로 출력되는가

코드 리뷰, 이렇게 질문해보세요

무엇을 물어야 할지 모르겠다면 아래 질문을 그대로 사용해도 좋습니다.

상황	질문 예시
서비스 이름이 낯설 때	. “이 서비스 이름은 왜 이렇게 지었어요?”
environment 값이 다를 때	. “이 값은 어디서 가지고 온 거예요?”
구조가 다르게 짜여 있을 때	. “이 방식으로 하면 어떤 점이 더 편해요?”

오늘의 코드 리뷰 방식

 **어제와 같은 방식입니다 — 옆 자리 동료와 함께, 체크리스트로.**

- 이번엔 어제 만든 Dockerfile들을 그대로 재사용하고, docker-compose.yml만 새로 작성합니다.
- 옆자리 동료와 서로 다른 방식으로 작성했다면, 그 차이를 비교해보는 것도 좋은 공부입니다.

종합실습 2 목표

🎯 *어제 만든 3-tier 미니 게시판을, 오늘 배운 **Docker Compose**로 다시 완성*

- 1일차 : docker run을 세 번, docker network create도 따로 실행
- 2일차 : **docker-compose.yml** 파일 하나 + **docker compose up** 한 줄

아키텍처 다시 보기

*어제와 똑같은 3-tier 구조를, 오늘은 **Compose** 파일 하나로 표현*



Web

nginx로 화면 조회 서비스



WAS

요청 처리하는 서버 서비스

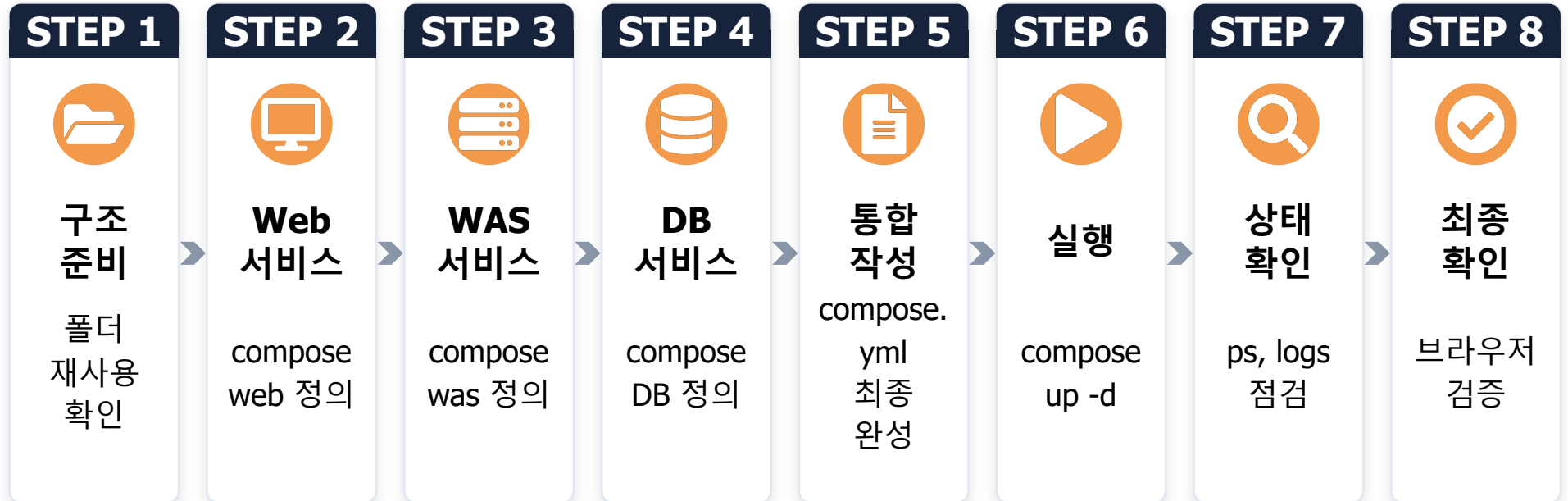


DB

데이터 저장 PostgreSQL

진행 순서 로드맵

여덟 단계로 나누어 차근차근 진행합니다.



[Step 1] 어제 폴더 재사용 확인하기

🕒 어제 만든 **board** 폴더를 그대로 사용합니다.

- ❶ `cd board` 로 어제 만든 폴더로 이동한다
- ❷ `ls -al` 로 `web`, `was` 폴더와 각각의 `Dockerfile`이 그대로 있는지 확인한다
- ❸ 어제 실행해둔 컨테이너가 있다면 `docker stop`, `docker rm`으로 정리한다

[Step 1] 예상 화면

*어제 작성한 **web/Dockerfile**을 그대로 **build** 항목에 연결합니다.*

```
$ cd board
$ ls -al
drwxr-xr-x 4 user  staff  128  8월 19 09:00 was
drwxr-xr-x 4 user  staff  128  8월 19 09:00 web

$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   STATUS   PORTS   NAMES
(어제 컨테이너가 남아있다면 여기 표시됨 → docker rm -f 로 정리)
```


[Step 2] Web 서비스 정의하기

*어제 작성한 **web/Dockerfile**을 그대로 **build** 항목에 연결합니다.*

```
services:  
  web:  
    build: ./web  
    ports:  
      - "8080:80"  
    depends_on:  
      - was
```

[Step 3] WAS 서비스 정의하기

*어제 -e로 직접 넣었던 환경 변수를 **environment** 항목으로 옮깁니다.*

```
was:
  build: ./was
  ports:
    - "5001:5000"
  environment:
    - DB_HOST=db
  depends_on:
    - db
```

💡 **DB_HOST=db** — db는 아래에서 정의할 서비스 이름과 정확히 같아야 합니다.

[Step 4] DB 서비스 정의하기

*공식 **postgres** 이미지와, 데이터를 지킬 볼륨을 함께 적습니다.*

```
db:
  image: postgres:17
  environment:
    - POSTGRES_PASSWORD=1234
    - POSTGRES_DB=board
  volumes:
    - board-data:/var/lib/postgresql/data

volumes:
  board-data:
```

💡 PostgreSQL의 데이터 저장 경로는 **/var/lib/postgresql/data**

[Step 5] docker-compose.yml, 전체 합치기

```
services:
  web:
    build: ./web
    ports: ["8080:80"]
    depends_on: [was]
  was:
    build: ./was
    ports: ["5000:5000"]
    environment:
      - DB_HOST=db
    depends_on: [db]
  db:
    image: postgres:17
    environment:
      - POSTGRES_PASSWORD=1234
      - POSTGRES_DB=board
    volumes: ["board-data:/var/lib/postgresql/data"]

volumes:
  board-data:
```

💡 서비스가 한 파일 안에
나란히 정리된 모습

[Step 6] 실행하기

🕒 **완성한 파일로 세 서비스를 한 번에 실행합니다.**

- ❶ docker compose build 로 web·was 이미지를 새로 빌드한다
- ❷ docker compose up -d 로 세 서비스를 모두 실행한다
- ❸ 터미널에 지나가는 로그를 보며 db → was → web 순서로 켜지는지 관찰한다

[Step 6] 예상화면



```
$ docker compose build
[+] Building 4.8s (16/16) FINISHED
=> [web internal] load build definition from Dockerfile
=> [web 1/2] FROM docker.io/library/nginx:alpine
=> [web 2/2] COPY index.html /usr/share/nginx/html/index.html
=> [was internal] load build definition from Dockerfile
=> [was 1/4] FROM docker.io/library/python:3.10-slim
=> [was 2/4] WORKDIR /app
=> [was 3/4] RUN pip install --no-cache-dir -r requirements.txt
=> [was 4/4] COPY . .
=> exporting to image
```

[Step 6] 예상화면



```
$ docker compose up -d  
[+] Running 4/4  
✓ Network board_default Created  
✓ Container board-db-1 Started  
✓ Container board-was-1 Started  
✓ Container board-web-1 Started
```

[Step 7] 상태 확인하기

🕒 세 서비스가 모두 정상 실행됐는지 점검합니다.

- ❶ `docker compose ps` 로 세 서비스 모두 Up 상태인지 확인한다
- ❷ `docker compose logs -f was` 로 DB 연결 로그를 확인한다
- ❸ 문제가 있다면 `docker compose logs -f db` 도 함께 확인한다

`depends_on`은 "순서"만 보장하지, "준비 완료"는 보장하지 않습니다

[Step 7] 예상화면



```
$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	STATUS	PORTS
board-db-1	postgres:17	"docker-entrypoint.s..."	db	Up 10 seconds	5432/tcp
board-was-1	board-was	"python app.py"	was	Up 10 seconds	0.0.0.0:5001->5000/tcp
board-web-1	board-web	"/docker-entrypoint...."	web	Up 10 seconds	0.0.0.0:8080->80/tcp

```
$ docker compose logs -f was
```

```
board-was-1 | * Serving Flask app 'app'  
board-was-1 | * Running on http://0.0.0.0:5000  
board-was-1 | [DB 연결 성공] db:5432 접속 완료
```

```
$ docker compose logs -f db
```

```
board-db-1 | LOG: database system is ready to accept connections
```

[Step 8] 브라우저로 최종 확인하기

🕒 어제와 똑같은 화면이, 훨씬 적은 명령어로 완성됐는지 확인합니다.

- ❶ 브라우저에서 `http://localhost:8080` 에 접속한다
- ❷ 글쓰기를 시도해 정상적으로 저장·조회되는지 확인한다
- ❸ `docker compose down` 후 다시 `up` 해서, 볼륨 덕분에 데이터가 남아있는지 확인한다

[Step 8] 예상화면

```
$ docker compose down
[+] Running 4/4
✓ Container board-web-1   Removed
✓ Container board-was-1   Removed
✓ Container board-db-1    Removed
✓ Network board_default   Removed

$ docker compose up -d
[+] Running 4/4
✓ Network board_default   Created
✓ Container board-db-1    Started
✓ Container board-was-1   Started
✓ Container board-web-1   Started
```

브라우저

http://localhost:8080 접속

→ 게시판 화면 정상 표시

글쓰기 테스트 → 목록에 정상 반영됨 확인

브라우저 새로고침

→ 조금 전 작성한 글이 그대로 남아있음

→ **volumes(board-data)** 덕분에
컨테이너를 지웠다 다시 만들어도
데이터가 유지됨을 확인

코드 리뷰 체크리스트

옆 동료의 `docker-compose.yml`을 함께 확인해보세요.

- ☐ services 아래 들여쓰기가 일관되게 되어 있는가
- ☐ 각 서비스의 build 경로 또는 image 이름이 정확한가
- ☐ depends_on 순서가 실제 의존 관계와 맞는가
- ☐ environment의 DB_HOST 값이 db 서비스 이름과 일치하는가
- ☐ volumes가 최상위와 서비스 내부 양쪽에 모두 선언되어 있는가

친해지길 바래3~



옆자리 동료와 이야기해보기

자리를 바꿔 앉아, 서로의 **docker-compose.yml**을 체크리스트로 리뷰해보세요.



10분

제출물

소스 파일

- docker-compose.yml 1개
(web/was Dockerfile은 1일차
재사용이므로 생략 가능)

실행 증빙 자료

- 실행결과 화면 Capture
 - docker ps 결과 (컨테이너 3개)
+ 브라우저 화면
- 동료와의 코드 리뷰 내용 정리

제출 기한 및 제출 방법

- 캠퍼스_반_이름_실습명.zip으로 제출 (판교캠퍼스_5반_홍길동_day2종합실습.zip)
 - 동료와의 코드리뷰 내용을 정리하여 PDF 파일로 생성
 - 소스 파일 + PDF 파일 구성으로 압축해서 제출
- 기한 : 과정 종료 시까지 (~21:00 제출 접수, 제출 기한 초과 시 감점 처리 예정)

자주 발생하는 오류 (1) — 연결 문제

증상	점검 사항
WAS 로그에 DB 연결 실패	DB_HOST를 localhost로 적지 않았는지 확인 (서비스 이름 db여야 함)
DB가 준비되기 전에 WAS가 먼저 시도	- depends_on은 순서만 보장 - 완전히 준비될 때까지 기다려주진 않습니다

자주 발생하는 오류 (2) — 문법 문제

증상	점검 사항
파일을 읽지 못한다는 오류	들여쓰기에 탭이 섞여있지 않은지 확인 (스페이스만 사용)
코드를 고쳤는데 반영이 안 됨	docker compose build 후 up 해야 새 이미지가 반영됩니다
포트 충돌 오류	8080, 5000번을 다른 프로그램이 이미 쓰고 있지 않은지 확인

종합 실습 2 셀프 점검

아래 항목에 모두 체크할 수 있다면 이틀간의 실습을 완주한 것입니다.

- ☐ docker-compose.yml을 처음부터 직접 작성했다
- ☐ docker compose up -d 로 세 서비스를 한 번에 실행했다
- ☐ 어제와 오늘의 차이(수동 vs Compose)를 설명할 수 있다
- ☐ 볼륨 덕분에 데이터가 유지되는 것을 직접 확인했다
- ☐ 동료와 서로의 코드를 리뷰했다

[Quiz]

Compose 파일에서 이미 존재하는 이미지를 그대로 쓸 때 사용하는 항목은?

- ☐ build
- ☐ image
- ☐ ports
- ☐ depends_on

[Quiz]

Compose 파일에서 이미 존재하는 이미지를 그대로 쓸 때 사용하는 항목은?

☐ build

☒ **image**

☐ ports

☐ depends_on



image는 이미 만들어진 이미지를 그대로 가져와 쓸 때 사용.
우리가 만든 **db** 서비스가 이 방식

종합 실습 2 정리

✓ 1일차 Dockerfile을 그대로 재사용

✓ docker compose up 한 번으로 전체 서비스를 실행

✓ 이제 어떤 다중 컨테이너 앱도 스스로 컨테이너화 가능

✓ docker-compose.yml 하나로 3-tier 앱을 정의

✓ 네트워크가 자동으로 구성되는 것을 확인

Day 2 마무리 이야기

이들간의 여정을 팀과 함께 돌아봅시다.

docker-compose.yml을 작성하며 가장 편하다고 느낀 부분은 무엇이었나요?

1일차(수동 연결)와 2일차(**Compose**)을 비교했을 때 가장 크게 느낀 차이는?

이번 이틀 동안 가장 유용했던 비유는 무엇이었나요?

다음 과정(쿠버네티스)에서는 오늘의 **Compose**가 어떻게 확장될지 예상해봅시다.

학습 요약 & 다음 과정



이번 과정에서 배운 것

- Linux 기초와 터미널 사용법
- 컨테이너·이미지 개념
- Dockerfile 작성과 이미지 빌드
- 포트·볼륨으로 연결과 저장 관리
- Docker Compose로 3-tier 앱 완성
- 짝과 함께하는 코드 리뷰 경험



다음 과정 — 쿠버네티스 이해

- 컨테이너 수가 늘어나면 왜 어려워지는가
- 여러 서버에 걸쳐 자동으로 배포, 확장하는 법
- 오늘 만든 Compose 구성이 어떻게 확장되는지
- Pod, Service, Deployment 등 핵심 개념

SKALA 3) Cloud – 2. 컨테이너 이해 및 애플리케이션 컨테이너화

[Appendix] 단어 사전



API

“ 비유로 말하면

식당의 주문 창구입니다. 주방에 직접 안 들어가도, 창구에 주문하면 음식이 나옵니다.

☰ 정확히 말하면

프로그램끼리 정해진 방식으로 요청·응답을 주고받는 약속(인터페이스)입니다.

오케스트레이션



오케스트레이션

“ 비유로 말하면

지휘자가 여러 연주자를 조율하듯, 여러 컨테이너를 자동으로 관리하는 것입니다.

☰ 정확히 말하면

여러 컨테이너의 배포·확장·복구를 자동으로 관리하는 것. 다음 과정의 핵심 주제입니다.

매니페스트 (Manifest)



매니페스트 (Manifest)

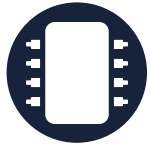
“ 비유로 말하면

이사 갈 때 적는 짐 목록표입니다.

☰ 정확히 말하면

무엇을 어떻게 배포할지 적어둔 설정 파일입니다. Compose 파일도 일종의 매니페스트입니다.

커널 (Kernel)



커널 (Kernel)

“ 비유로 말하면

운영체제의 심장입니다.

☰ 정확히 말하면

운영체제의 가장 핵심적인 부분으로, 하드웨어를 직접 관리합니다.

데몬 (Daemon)



데몬 (Daemon)

“ 비유로 말하면

보이지 않는 곳에서 항상 일하는 집사입니다.

☰ 정확히 말하면

뒤에서 계속 실행되며 요청을 기다리는 프로그램입니다. Docker Daemon이 대표적입니다.

CI/CD



CI/CD

“ 비유로 말하면

자동으로 재료를 검수하고 요리까지 해주는 자동화 주방입니다.

☰ 정확히 말하면

코드 변경 사항을 자동으로 테스트하고 배포하는 과정입니다.



수고하셨습니다.

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.